

Jazyk C# pro začátečníky

Tato skripta jsou určena úplným začátečníkům v programování. Nevyžadují žádné předchozí znalosti. Každá kapitola obsahuje praktické příklady, vysvětlení a cvičení.

Obsah

1. Co je C# a k čemu slouží
2. Instalace Visual Studia
3. První program v C#
4. Datové typy a proměnné
5. Vstup a výstup
6. Operátory a výrazy
7. Ladění programu
8. Větvění programu (if, switch)
9. Cykly (for, while, do-while)
10. Pole
11. Funkce (metody)
12. Práce s řetězcí
13. Doporučené zásady psaní kódu

Kapitola 1: Co je C# a k čemu slouží

Stručná historie

Jazyk C# (čte se „sí šarp“) vyvinula firma Microsoft a poprvé vydala v roce 2002 jako součást platformy .NET. Název pochází z hudby: nota **C#** (Cis) je o půltón výše než nota C, čímž autoři naznačili, že jde o vylepšenou verzi jazyka C++.

C# vychází ze syntaxe jazyka C, který Dennis Ritchie napsal v roce 1972 pro operační systém Unix. Na C navázaly jazyky C++ (Bjarne Stroustrup, 1986) a Java (James Gosling, Sun Microsystems, 1996). C# pak převzal to nejlepší z obou a přidal vlastní vylepšení.

Klíčová vlastnost: Od verze .NET Core (2016), dnes označované jako .NET, je C# **multiplatformní** — programy psané v C# lze spustit na Windows, Linuxu i macOS. Do roku 2019 byl aktivně vyvíjen i .NET Framework, který běžel jen na Windows. Dnes již v něm nové věci nevznikají.

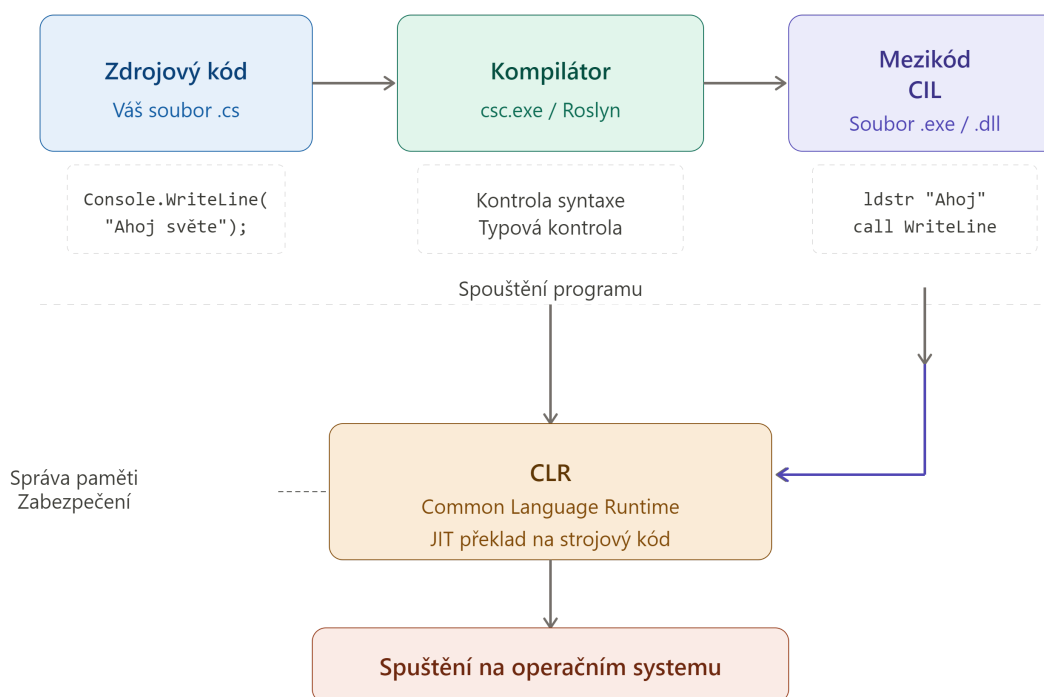
Kde se C# používá

Oblast	Technologie
Webový backend	ASP.NET Core, REST API
Desktopové aplikace	WinForms, WPF, MAUI

Oblast	Technologie
Hry	Unity herní engine
Cloudové aplikace	Azure, microservices
Mobilní aplikace	.NET MAUI

Platforma .NET

C# programy neběží přímo jako strojový kód. Kompilátor je přeloží do **mezikódu** (CIL — Common Intermediate Language), který pak při spuštění interpretuje prostředí .NET (CLR — Common Language Runtime). Výhodou je, že stejný kód může běžet na různých operačních systémech.



Vývojové prostředí (IDE)

Pro psaní C# programů potřebujeme **IDE** (Integrated Development Environment — integrované vývojové prostředí). IDE spojuje:

- textový editor se zvýrazňováním syntaxe,
- kompilátor (překládá kód),
- debugger (nástroj pro ladění),
- automatické doplňování kódu (IntelliSense).

Nejpoužívanější IDE pro C# je **Microsoft Visual Studio** a **Visual Studio Code** (VS Code), případně **JetBrains Rider**.

Shrnutí

- C# je moderní, bezpečný a výkonný jazyk vyvinutý firmou Microsoft.
- Je multiplatformní — spouští se na Windows, Linuxu i macOS.
- Používá se pro webový backend, desktopové aplikace, hry i mobilní vývoj.
- Pro vývoj v C# doporučujeme Visual Studio Community (zdarma).

Kapitola 2: Instalace a nastavení Visual Studia

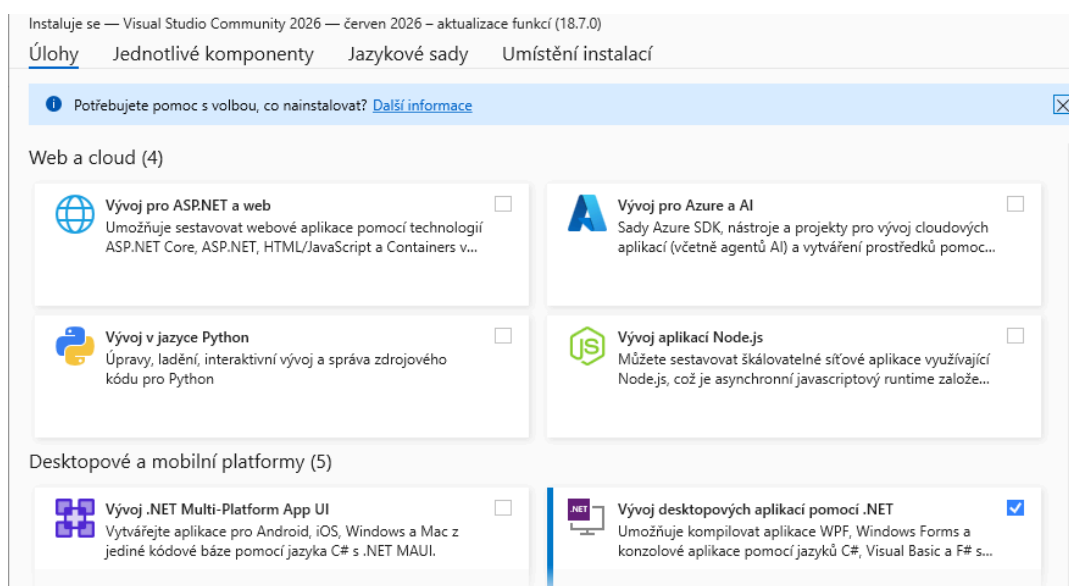
Co budeme potřebovat

- Počítač s Windows 10 nebo Windows 11
- Přibližně 6–10 GB volného místa na disku
- Připojení k internetu

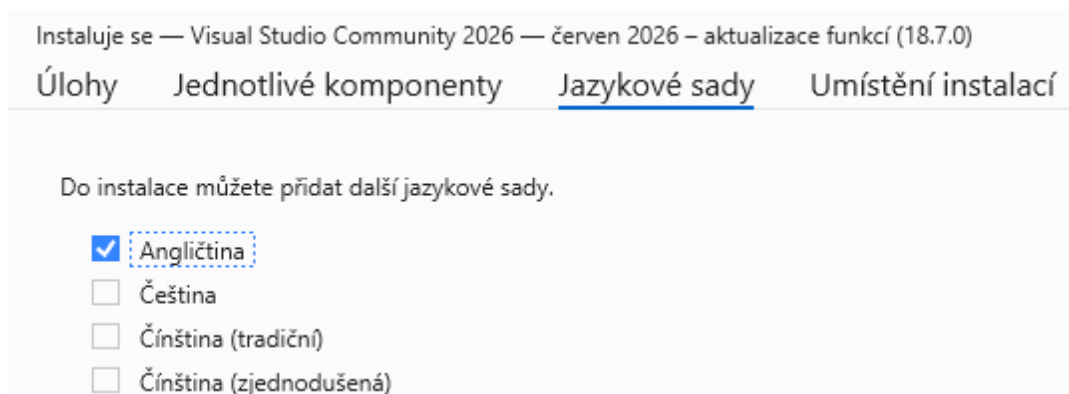
Alternativně lze na Linuxu nebo macOS použít **Visual Studio Code** nebo **JetBrains Rider** s rozšířením C#.

Stažení a instalace

1. Přejděte na stránku <https://visualstudio.microsoft.com/cs/downloads/>
2. Stáhněte verzi **Community** — je zdarma a plně funkční pro studijní i osobní účely.
3. Spustěte stažený instalátor **VisualStudioSetup.exe**.
4. Instalátor zobrazí výběr „sad funkcí“ (Workloads). Zvolte: **Vývoj desktopových aplikací pomocí .NET**.



5. Na záložce **Jazykové sady** doporučujeme nainstalovat angličtinu a odebrat češtinu — anglické prostředí usnadňuje vyhledávání řešení problémů na internetu.



6. Klikněte na **Nainstalovat** a počkejte na dokončení.

Proč angličtina? Naprostá většina výukových materiálů, dokumentace a komunitních fór (Stack Overflow, GitHub) je v angličtině. Pokud máte VS v češtině a potřebujete vyhledat řešení chyby, musíte nejprve přeložit název volby z češtiny — to zdržuje.

Pozdější přidání komponent

Pokud budete chtít doinstalovat další sady funkcí, spusťte znovu **Visual Studio Installer** (klávesa Windows, vyhledejte „Visual Studio Installer“), přihlaste se jako správce a klikněte na **Změnit**.

První spuštění Visual Studia

Po prvním spuštění po vás bude chtít přihlásit pomocí účtu. Pokud nechcete stačí dát **Skip and add accounts later** popřípadě se zaregistrujete nebo přihlaste. V dalším kroku můžeme vybrat motiv a Development settings (ten necháme **General). A spustíme pomocí **Start Visual Studio**

Vytvoření prvního projektu

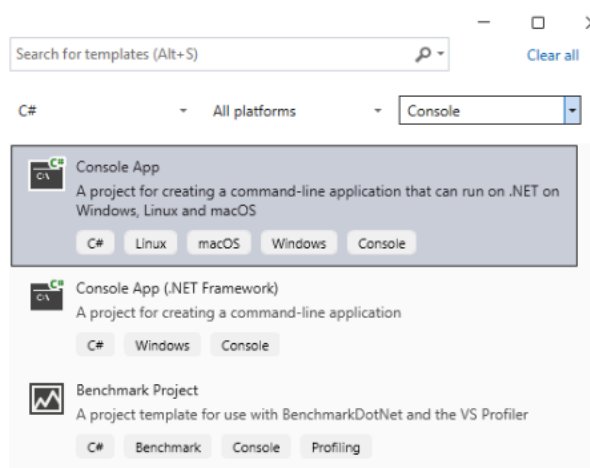
Po spuštění Visual Studia:

1. Klikněte na **Create a new project** (Vytvořit nový projekt).
2. V filtrovacích polích vyberte jazyk **C#** a typ projektu **Console**.
3. Zvolte šablonu **Console App** (horní z nabízených, ten bez závorky — jde o moderní .NET, ne starší .NET Framework).

Create a new project

Recent project templates

A list of your recently accessed templates will be displayed here.



4. Klikněte **Next** a zadejte název projektu, například **PrvniProgram**.
5. Umístění na disku ponechejte pokud nechcete uložit jinak.
6. Klikněte **Next** — jako verzi frameworku ponechte nejnovější nabízenou. **Důležité:** Zaškrtněte volbu **Do not use top-level statements**. Tím se zobrazí plná struktura programu včetně třídy a metody Main, která je pro začátečníky přehlednější.

Additional information

Console App

C#

Linux

macOS

Windows

Console

Framework ⓘ

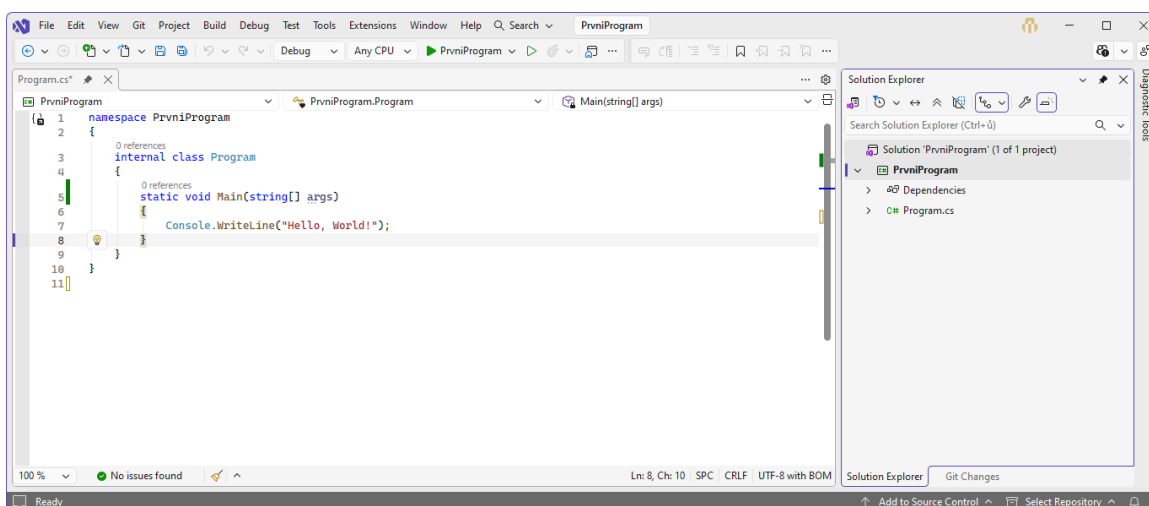
☒ Do not use top-level statements ⓘ

☐ Enable native AOT publish ⓘ

7. Klikněte **Create**.

Seznámení s prostředím Visual Studio

Visual Studio je velmi rozsáhlý nástroj s mnoha funkcemi. Není se ho třeba bát — pro začátek nám stačí znát jen čtyři základní věci: kde psát kód, kde najít soubory projektu, jak projekt otevřít nebo uložit a jak program spustit.



Hlavní okno Visual Studio se skládá z těchto částí:

Editor kódu (velká plocha uprostřed) — zde píšeme program. Po vytvoření nového projektu se zde automaticky otevře soubor **Program.cs** s připraveným kódem.

Solution Explorer (panel vpravo) — zobrazuje strukturu projektu. Vidíme zde název projektu a seznam souborů. Poklepáním na soubor ho otevřeme v editoru. Někdy ho překrývá okno copilota. Stačí jen klepnout na "ouško" Solution Explorer pro zobrazení.

Panel nabídek (nahore) — obsahuje veškeré příkazy. Pro začátek nás zajímají:

- **File** → **Open** → **Project/Solution** — otevření existujícího projektu
- **File** → **Save** nebo klávesa **Ctrl+S** — uložení

Poznámka: Visual Studio soubor automaticky uloží při každém spuštění programu (**Ctrl+F5** nebo **F5**), takže na ruční uložení nemusíte myslet při každé změně. Přesto se **Ctrl+S** hodí — například před zavřením editoru nebo před sdílením souboru.

Tlačítko spuštění (zelená šipka s názvem projektu v horním panelu, například ► **PrvniProgram**) — spustí program s debuggerem. Pohodlnější je klávesová zkratka **F5**.

Doporučení: Používejte **F5** — spustí program s debuggerem, takže máte kdykoliv možnost program zastavit a zkoumat hodnoty proměnných. Pokud se okno konzole po skončení programu ihned zavře, přidejte na konec programu **Console.ReadKey()** — program počká na stisk klávesy.

S dalšími částmi prostředí se budeme seznamovat postupně, až je budeme potřebovat.

Struktura solution a projektu

Visual Studio pracuje s hierarchií: **solution** → **projekt** → **soubory (.cs)**.

- **Solution** (řešení) je „složka složek“ — sdružuje projekty, které spolu souvisí. Hlavním souborem je **.slnx** (nebo starší **.sln**), který obsahuje jen seznam obsažených projektů a cesty k nim — žádný kód v něm není.

- **Projekt** odpovídá jednomu spustitelnému programu (výsledkem překladu je soubor `.exe` nebo `.dll`). Hlavním souborem je `.csproj`, který popisuje, jak projekt sestavit — jaké soubory do něj patří, jakou verzi .NET používá a jaké knihovny vyžaduje.
- **Soubory** `.cs` obsahují zdrojový kód v jazyce C#.

Při vytvoření nového projektu se automaticky vytvoří i solution se stejným názvem.

Praktická poznámka: Pokud chcete sdílet svůj projekt (například při žádosti o pomoc), nestačí poslat jen soubor `.sln` nebo `.csproj` — žádný kód v nich není. Pro jednoduchý program stačí poslat příslušné soubory `.cs`. U většího projektu sdílejte celou složku zabalenou do archivu `.zip` — pozor, některé e-mailové filtry přílohy `.zip` blokují, v takovém případě použijte nejlépe odkaz na úložiště (např. Google Drive nebo GitHub).

Nastavení klávesových zkratk

Visual Studio nabízí několik sad klávesových zkratk. Abychom v kurzu měli všichni stejné, nastavte sadu **Visual C# 2005**:

1. Otevřete **Tools** → **Options**.
2. Přejděte na **Environment** → **Keyboard**.
3. V rozbalovacím seznamu **Apply the following additional keyboard mapping scheme** vyberte **Visual C# 2005**.
4. Klikněte **OK**.

Nejdůležitější zkratky, které budeme používat:

Zkratka	Akce
Ctrl+F5	Spustit program bez debuggeru
F5	Spustit program s debuggerem
Ctrl+S	Uložit soubor
Ctrl+K, Ctrl+C	Zakomentovat vybraný kód
Ctrl+K, Ctrl+U	Odkomentovat vybraný kód
F2	Přejmenovat proměnnou/metodu všude v projektu
Ctrl+K, Ctrl+D	Přeformátovat celý soubor

Shrnutí

- Visual Studio Community je zdarma a plně postačuje pro výuku i osobní projekty.
- Při instalaci zvolíme sadu funkcí „Vývoj desktopových aplikací pomocí .NET“.
- Projekty se vytvářejí přes **File / New Project** nebo **Create a new project** na úvodní obrazovce.

Kapitola 3: První program v C#

Struktura programu

Po vytvoření nového projektu Visual Studio automaticky vygeneruje soubor **Program.cs** a otevře ho v editoru. Tento soubor obsahuje připravený kód, který lze rovnou spustit — zkuste **Ctrl+F5** ještě předtím, než cokoliv změníte:

```
namespace PrvniProgram;

internal class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Hello, World!");
    }
}
```

A nyní rozeberme jednotlivé části:

Část	Vysvětlení
<code>namespace PrvniProgram</code>	Jmenný prostor — logická skupina tříd. Zabraňuje kolizi názvů v rozsáhlých projektech.
<code>internal class Program</code>	Definice třídy. V C# musí být veškerý kód uvnitř třídy.
<code>static void Main(string[] args)</code>	Vstupní bod programu. Program vždy začíná zde.
<code>Console.WriteLine(...)</code>	Příkaz pro výpis textu na obrazovku (konzole = příkazový řádek, takové to černé okno).

Tip: Pokud vám závorky připadají nepřehledné, lze řádek s `namespace` zakončit středníkem a celý blok závorek vynechat. Kód bude o jednu úroveň odsazení plošší:

```
namespace PrvniProgram;

internal class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Hello, World!");
    }
}
```



```
}
}
```

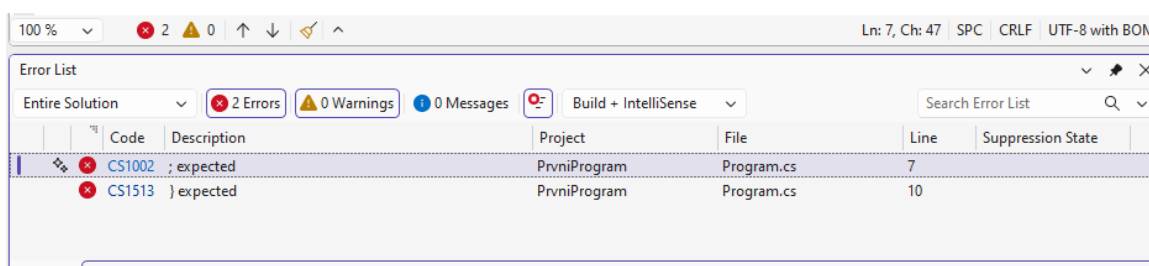
Obě varianty jsou správně — je to jen otázka stylu.

Tip — snippets: Až budete psát nový soubor od začátku, snippets vám ušetří psaní: `class` + Tab Tab vloží prázdnou třídu, `svm` + Tab Tab vloží celou metodu `static void Main(string[] args)`.

Časté chyby začátečníků:

- Chybějící středník na konci příkazu (;) — v C# ukončuje každý příkaz.
- Záměna velkých a malých písmen — C# rozlišuje `main` a `Main`. Správně je `Main`.
- Nepárové složené závorky {} — každá otevírací závorka musí mít svou uzavírací.

Již při psaní kódu lze zobrazit syntaktické chyby a hned je opravit. Zobrazení Error listu lze poklepnutím na Errors ve stavové liště nebo v menu liště **View - Error List**.



Spuštění programu

- **Ctrl+F5** — spustí program bez debuggeru (okno konzoly zůstane otevřené).
- **F5** — spustí program s debuggerem (pro ladění).

Speciální znaky v textu (escape sekvence)

Některé znaky v textu nelze napsat přímo. Používají se tzv. **escape sekvence** — dvojice znaků začínající zpětným lomítkem:

Escape sekvence	Výsledek
<code>\n</code>	Nový řádek (newline)
<code>\t</code>	Tabelátor — odsazení (tab)
<code>\\</code>	Zpětné lomítko
<code>\"</code>	Uvozovka uvnitř řetězce

Příklad:

```
Console.WriteLine("Jméno:\tJan Novák\nVěk:\t25");
```

Výstup:

```
Jméno:  Jan Novák
Věk:    25
```

Komentáře

Komentáře jsou části kódu, které kompilátor ignoruje a slouží především k vysvětlení logiky pro programátora. Jsou velmi důležité – je lepší mít komentářů více než žádné. Pomohou vám zejména ve chvíli, kdy se ke kódu po čase vrátíte, protože rychle připomenou, co program nebo jeho jednotlivé části dělají. Jejich význam je ještě větší při čtení cizího kódu.

```
// Jednořádkový komentář – platí do konce řádku

/* Víceřádkový komentář
   může zabírat více řádků */

Console.WriteLine("Ahoj!"); // Komentář může být i za příkazem
```

Jak psát kod

Při psaní kódu je nutné dodržovat stanovená pravidla a zásady, které ovlivňují jeho správnost, čitelnost i přehlednost. Správně zapsaný kód není důležitý pouze pro fungování programu, ale také pro jeho další úpravy, opravy a sdílení s ostatními programátory.

Se zásadami psaní kódu se budeme seznamovat postupně v jednotlivých kapitolách, kde budou vždy uvedena konkrétní pravidla vztahující se k probíranému tématu. Souhrn všech doporučených pravidel a zásad je uveden v kapitole 13, která slouží jako celkové shrnutí.

Dodržování těchto pravidel je základním předpokladem pro tvorbu kvalitního a přehledného kódu.

Shrnutí

- Každý program v C# musí mít metodu `Main` — odtud program začíná.
- Příkazy se ukončují středníkem `;`.
- Bloky kódu jsou uzavřeny do složených závorek `{}`.

Více příkladů v jednom projektu

Při sledování videí a práci na cvičeních budete postupně přidávat nové příklady. Protože v jednom projektu může existovat pouze jedna metoda **Main** (ta řídí celý program), používáme v kurzu následující postup:

- Jeden soubor **Program.cs** obsahuje **řídící třídu** s metodou **Main**, která volá příklady z ostatních tříd.
- Každý nový příklad vytvoříte v **samostatné třídě** v novém souboru **.cs**. Pomocí snippetu **svm** + Tab Tab vygenerujete metodu, kterou přejmenujete z **Main** na **Mainx** (nebo jiný název — **MainDruhy**, **MainTriangle** apod.).
- V řídící třídě pak zavoláte příslušnou metodu.

```
// Program.cs – řídící třída
namespace MujProjekt;

internal class Program
{
    static void Main(string[] args)
    {
        Prvni.Mainx();
        // Druhy.Mainx(); // Zakomentujte příklady, které nechcete
        // spouštět
    }
}
```

```
// Prvni.cs – první příklad
namespace MujProjekt;

internal class Prvni
{
    public static void Mainx()
    {
        Console.WriteLine("Toto je první příklad.");
    }
}
```

Nový soubor **.cs** přidáte přes **Project** → **Add New Item** → **Class** (nebo **Ctrl+Shift+A**).

Kapitola 4: Datové typy a proměnné

Co je proměnná

Proměnná je pojmenované místo v paměti, kde program uchovává hodnotu. Každá proměnná má:

- **název** (identifikátor) — například **vek**, **jmeno**, **cena**,

- **datový typ** — určuje, jaký druh hodnoty může proměnná uchovávat,
- **hodnotu** — aktuální obsah proměnné.

Pravidla pro název proměnné:

- Může obsahovat písmena, číslice a podtržítka `_`.
- Nesmí začínat číslicí.
- Nesmí být klíčové slovo jazyka (např. `int`, `if`, `for`).
- C# rozlišuje velká a malá písmena: `Vek` a `vek` jsou dvě různé proměnné.
- Konvence: proměnné se pojmenovávají `camelCase` — první slovo malým, každé další velkým písmenem: `platZamestnanec`.

Deklarace a přiřazení

```
// Deklarace proměnné (vytvoření)
int vek;

// Přiřazení hodnoty
vek = 25;

// Deklarace a přiřazení můžeme udělat najednou (inicializace)
int rok = 2025;
string jmeno = "Jan";
double cena = 99.90;
bool jeStudent = true;
```

Doporučení: Proměnnou vždy inicializujte hned při deklaraci. Deklarace bez přiřazení hodnoty (`int vek;`) je sice syntakticky správně, ale snadno vede k chybě — pokud proměnnou omylem použijete před přiřazením, kompilátor ohlásí chybu. Čitelnější a bezpečnější je vždy psát `int vek = 0;` nebo rovnou použít smysluplnou počáteční hodnotu.

Přehled základních datových typů

Celočíselné typy

Typ	Velikost	Rozsah	Kdy použít
<code>byte</code>	1 B	0 až 255	Malá kladná čísla (věk, procenta)
<code>short</code>	2 B	-32 768 až 32 767	Středně velká celá čísla
<code>int</code>	4 B	-2,1 mld až 2,1 mld	Výchozí volba pro celá čísla
<code>long</code>	8 B	$\pm 9,2 \cdot 10^{18}$	Velmi velká čísla
<code>uint</code>	4 B	0 až 4,3 mld	Kladná celá čísla

Doporučení: Pro celá čísla používejte standardně `int`. Typ `byte` nebo `short` volte jen tehdy, když skutečně šetříte paměť (například ve velkých polích).

Desetinné typy

Typ	Velikost	Přesnost	Kdy použít
<code>float</code>	4 B	7 číslic	Grafika, fyzikální simulace
<code>double</code>	8 B	15 číslic	Výchozí volba pro desetinná čísla
<code>decimal</code>	16 B	28 číslic	Finanční výpočty (ceny, daně)

Přesnost udává, kolik **platných číslic** dokáže typ spolehlivě uchovat. Zbytek je zaokrouhlen:

```
float f = 1.23456789f;           // uloží přibližně 1.234568
// přesnost 7 číslic
double d = 1.23456789012345678; // uloží přibližně
1.2345678901235 – přesnost 15 číslic
decimal m = 1.234567890123456789m; // uloží přesně 28
číslíc
```

Upozornění: Desetinná čísla typu `float` a `double` nejsou přesná — jsou uložena v binárním formátu, který ne vždy přesně odpovídá desetinnému zápisu. Chyba zaokrouhlení se může při opakovaných výpočtech postupně hromadit. Pro peníze vždy používejte `decimal`.

Typ `bool` (logická hodnota)

Uchovává pouze dvě hodnoty: `true` (pravda) nebo `false` (nepravda).

```
bool jeStudent = true;
bool jeZletily = false;
```

Typ `char` (znak)

Uchovává jeden znak (písmeno, číslici, interpunkci). Hodnota se zapisuje v **apostrofech**.

```
char pismeno = 'A';
char cislice = '5'; // Znak '5', nikoliv číslo 5
char mezera = ' ';
```

Časté chyba: Záměna apostrofů a uvozovek. Znaky (`char`) se píší do apostrofů `'A'`, řetězce (`string`) do uvozovek `"Ahoj"`.

Typ `string` (řetězec)

Uchovává posloupnost znaků — text libovolné délky.

```
string jmeno = "Jan Novák";
string prazdny = ""; // Prázdný řetězec
string viceRadku = "Řádek 1\nŘádek 2";
```

Pro zvidavé: Přehled datových typů C# a CLR

C# typ	Celé jméno	Velikost	Rozsah / Poznámka
<code>sbyte</code>	<code>SByte</code>	1 B	-128 až 127
<code>byte</code>	<code>Byte</code>	1 B	0 až 255
<code>short</code>	<code>Int16</code>	2 B	-32 768 až 32 767
<code>ushort</code>	<code>UInt16</code>	2 B	0 až 65 535
<code>int</code>	<code>Int32</code>	4 B	±2,1 miliardy
<code>uint</code>	<code>UInt32</code>	4 B	0 až 4,3 miliardy
<code>long</code>	<code>Int64</code>	8 B	$\pm 9,2 \cdot 10^{18}$
<code>ulong</code>	<code>UInt64</code>	8 B	0 až $1,8 \cdot 10^{19}$
<code>float</code>	<code>Single</code>	4 B	$\pm 3,4E+38$, přesnost 7 číslic
<code>double</code>	<code>Double</code>	8 B	$\pm 1,8E+308$, přesnost 15 číslic
<code>decimal</code>	<code>Decimal</code>	16 B	$\pm 7,9E+28$, přesnost 28 číslic
<code>bool</code>	<code>Boolean</code>	1 B	<code>true</code> nebo <code>false</code>
<code>char</code>	<code>Char</code>	2 B	Jeden znak Unicode

Klíčová slova jako `int`, `double` nebo `string` jsou zkrácené aliasy — pohodlnější zkratky za celá jména jako `Int32`, `Double` nebo `String`. V kódu lze použít obojí, v praxi se ale vždy používají kratší aliasy.

Datová konverze

Implicitní konverze (automatická)

Probíhá tehdy, když konvertujeme na „širší“ typ — tedy typ, který zabírá více bajtů a pojme větší rozsah hodnot. Kompilátor to provede sám, protože nehrozí ztráta dat. Představte si to jako přelití vody z menší nádoby do větší — vždy se vejde.

```
int i = 100;
long l = i;      // Automaticky: int → long, bezpečné
double d = i;    // Automaticky: int → double, bezpečné
```

Hierarchie automatické konverze (šipka znamená „lze přiřadit bez přetypování“):

byte → short → int → long → float → double

```
byte b = 10;
int i = b;      // byte se vejde do int
double d = i;   // int se vejde do double
// Nelze opačně bez přetypování:
// byte b2 = i; // Chyba kompilace!
```

Explicitní konverze (přetypování)

Nutná při konverzi na „užší“ typ — hrozí ztráta dat. Kompilátor nám to záměrně nedovolí udělat omylem, musíme mu výslovně říct, že víme, co děláme. Zapisuje se uvedením cílového typu v závorce před hodnotou.

```
double d = 9.99;
int i = (int)d;      // Ořízne desetinnou část — i == 9, ne 10!

int velke = 300;
byte b = (byte)velke; // 300 se nevejde do byte (max 255) — dojde k přetečení, b == 44
```

Časté chyby začátečníků:

- Přetypování pomocí `(int)` pouze ořízne desetinnou část — **nezaokrouhluje**. Pro zaokrouhlení použijte `Math.Round()`.
- Přetypování do menšího typu může způsobit **přetečení** bez varování — kompilátor neohlásí chybu, výsledek bude tiše nesprávný a za běhu programu se zobrazí nebo uloží nesmyslná hodnota.

Konverze řetězce na číslo

```
string text = "42";
int cislo = int.Parse(text);           // Vyvolá výjimku, pokud text
není číslo
int cislo2 = Convert.ToInt32(text); // Alternativní zápis – výsledek
je stejný

double realne = double.Parse("3,14"); // Na českém Windows
desetinná čárka
```

Poznámka: `int.Parse()` a `Convert.ToInt32()` dělají totéž — převádějí řetězec na číslo. V praxi uvidíte častěji `int.Parse()`, ale obě varianty jsou správně.

Pozor na desetinný oddělovač: Metoda `double.Parse` respektuje nastavení jazyka operačního systému. Na českém Windows je desetinným oddělovačem **čárka**, takže `double.Parse("3.14")` vyvolá výjimku — správně by bylo `double.Parse("3,14")`.

Konverze čísla na řetězec

```
int vek = 25;
string text = vek.ToString();           // "25"
string text2 = vek.ToString("D5"); // "00025" – s úvodními nulami
```

Pro zvědavé: Typ `var` — odvozený typ

Od C# 3.0 lze použít klíčové slovo `var`. Kompilátor sám odvodí typ z přiřazené hodnoty.

```
var cislo = 42;           // kompilátor odvodí int
var text = "Ahoj";        // kompilátor odvodí string
var cena = 9.99;          // kompilátor odvodí double
```

`var` neznamena „libovolný typ“ — typ je pevně určen při kompilaci z přiřazené hodnoty a nelze jej změnit. Používejte `var` tam, kde je typ z kontextu zřejmý.

Konstanty

Pokud se hodnota nesmí změnit, použijeme klíčové slovo `const`:

```
const double Pi = 3.14159265358979;
const int MaxPocetHracu = 4;
```

Pokus o změnu konstanty způsobí chybu kompilace.

Shrnutí

- Každá proměnná má název, typ a hodnotu.
- Pro celá čísla používejte `int`, pro desetinná `double`, pro text `string`, pro jeden znak `char`, pro logické hodnoty `bool`.
- Pro finanční výpočty vždy `decimal`.
- Automatická konverze probíhá z menšího na větší typ; opačně je nutné explicitní přetypování.

Kapitola 5: Vstup a výstup

Výstup na konzoli

Dva základní příkazy pro výstup:

```
Console.WriteLine("Text s odřádkováním");  
Console.Write("Text bez odřádkování");
```

Formátování výstupu

Zformátovat výstup lze několika způsoby. Doporučujeme však interpolaci řetězce (způsob 3) — je nejčitelnější a nejméně náchylná na chyby.

Způsob 1: Zřetězení pomocí +

```
string jmeno = "Jana";  
int vek = 22;  
Console.WriteLine("Jméno: " + jmeno + ", věk: " + vek);
```

Způsob 2: Indexované zástupné symboly (méně časté, starší styl)

```
Console.WriteLine("Jméno: {0}, věk: {1}", jmeno, vek);  
// {0} se nahradí první proměnnou, {1} druhou atd.
```

Způsob 3: Interpolace řetězce (doporučeno, moderní)

```
Console.WriteLine($"Jméno: {jmeno}, věk: {vek}");  
// Uvnitř {} lze použít libovolný výraz  
Console.WriteLine($"Za rok bude mít {jmeno} věk: {vek + 1}");
```

Formátování čísel

```
double cena = 1234.5;
Console.WriteLine($"{cena:F2}");           // 1234,50 – dvě desetinná
místa
Console.WriteLine($"{cena:N0}");           // 1 235 – celé číslo s
oddělovačem tisíců
Console.WriteLine($"{cena:C}");           // formát měny dle nastavení
OS
Console.WriteLine($"{0.75:P0}");           // 75% – procentuální formát
```

Vstup z klávesnice

```
Console.Write("Zadejte vaše jméno: ");
string jmeno = Console.ReadLine();         // Přečte celý řádek jako
string

Console.WriteLine($"Ahoj, {jmeno}!");
```

Čtení čísel ze vstupu

`Console.ReadLine()` vždy vrací `string`. Pro práci s číslem je nutné jej převést:

```
Console.Write("Zadejte váš věk: ");
string vstup = Console.ReadLine();
int vek = int.Parse(vstup);

// Kratší zápis na jeden řádek:
int vek2 = int.Parse(Console.ReadLine());
```

Poznámka: V novějších verzích .NET může kompilátor u `Console.ReadLine()` zobrazit varování, že výsledek může být `null`. Pro konzolový vstup to v praxi nenastane — varování lze ignorovat.

Časté chyby začátečníků:

- `int.Parse()` vyvolá výjimku (chybu za běhu), pokud uživatel nezadá platné číslo. Ošetření vstupů pomocí výjimek probereme v kurzu OOP.
- Záměna desetinné tečky a čárky: závisí na nastavení jazyka operačního systému. V českém Windows se jako desetinný oddělovač používá čárka.

Praktický příklad: Kalkulačka

```

namespace Kalkulacka;

internal class Program
{
    static void Main(string[] args)
    {
        // Vstup
        Console.Write("Zadejte první číslo: ");
        double a = double.Parse(Console.ReadLine());

        Console.Write("Zadejte druhé číslo: ");
        double b = double.Parse(Console.ReadLine());

        // Výpočet
        double soucet = a + b;
        double rozdil = a - b;
        double soucin = a * b;

        // Výstup
        Console.WriteLine($"{a} + {b} = {soucet}");
        Console.WriteLine($"{a} - {b} = {rozdil}");
        Console.WriteLine($"{a} * {b} = {soucin}");
        Console.WriteLine($"{a} / {b} = {a / b:F4}");
    }
}

```

Rozbor příkladu:

- Program přečte dvě čísla ze vstupu, provede čtyři základní operace a výsledky vypíše.
- Záměrně není ošetřeno dělení nulou. Dělení nulou způsobí chybu za běhu.
- `{a / b:F4}` formátuje výsledek na 4 desetinná místa.

Shrnutí

- `Console.WriteLine` a `Console.Write` slouží pro výstup.
- `Console.ReadLine()` přečte vstup jako text (string).
- Pro čtení čísla je nutné převést string na číselný typ pomocí `int.Parse()`, `double.Parse()` apod.
- Doporučujeme interpolaci řetězce (`$"..."`) pro čitelné formátování výstupu.

Kapitola 6: Operátory a výrazy

Aritmetické operátory

Operátor	Operace	Příklad	Výsledek
+	Sčítání	5 + 3	8

Operátor	Operace	Příklad	Výsledek
-	Odčítání	5 - 3	2
*	Násobení	5 * 3	15
/	Dělení	5 / 2	2 (celočíslné!)
%	Zbytek po dělení (modulo)	5 % 2	1

Důležité: Pokud jsou oba operandy celá čísla, provádí se **celočíslné dělení** — výsledek je celé číslo bez desetinné části. Pro desetinné dělení musí být alespoň jeden operand desetinný.

```
int a = 7, b = 2;

Console.WriteLine(a / b);    // 3  (celočíslné dělení)
Console.WriteLine(a % b);    // 1  (zbytek: 7 = 3*2 + 1)

// Desetinné dělení – stačí přidat .0 k jednomu operandu:
Console.WriteLine(a / 2.0);  // 3,5
Console.WriteLine((double)a / b); // 3,5 – přetypování
```

Operátor modulo — praktické využití

```
// Zjištění, zda je číslo sudé nebo liché
int n = 17;
if (n % 2 == 0)
    Console.WriteLine("Sudé");
else
    Console.WriteLine("Liché");

// Zjištění posledních dvou číslic roku
int rok = 2025;
Console.WriteLine(rok % 100); // 25
```

Zkrácené přiřazovací operátory

```
int x = 10;
x += 5;    // x = x + 5 => x je 15
x -= 3;    // x = x - 3 => x je 12
```

Operátory `+=` a `-=` jsou v praxi nejběžnější — setkáte se s nimi téměř v každém programu.

Pro zvědavé: Stejným způsobem fungují i operátory pro násobení, dělení a modulo:

```
x *= 2;    // x = x * 2  => x je 24
x /= 4;    // x = x / 4  => x je 6
x %= 4;    // x = x % 4  => x je 2
```

Inkrementace a dekrementace

```
int i = 5;
i++;    // i = i + 1  => i je 6
i--;    // i = i - 1  => i je 5
```

Postfixové operátory (`i++`, `i--`) jsou v praxi nejběžnější — typicky je uvidíte v cyklech.

Pro zvědavé: Existuje i tzv. prefixová varianta, kde operátor `++` či `--` stojí před proměnnou. Rozdíl se projeví jen tehdy, když operátor použijete uvnitř výrazu — prefixová varianta nejprve změní hodnotu, pak ji použije; postfixová naopak.

```
int a = 5;
int b = a++;    // Nejprve přiřadí (b = 5), pak zvýší (a = 6)
int c = ++a;    // Nejprve zvýší (a = 7), pak přiřadí (c = 7)
```

S prefixovými operátory se můžete setkat při čtení cizího kódu, sami je ale příliš potřebovat nebudete.

Doporučení pro začátečníky: Nepoužívejte `++` nebo `--` uvnitř složitějších výrazů. Píšte je vždy na samostatném řádku. Zabráníte tím obtížně odhalitelným chybám.

Relační (porovnávací) operátory

Výsledkem je vždy `bool` (true nebo false):

Operátor	Význam	Příklad
<code>==</code>	Rovná se	<code>a == b</code>
<code>!=</code>	Nerovná se	<code>a != b</code>
<code><</code>	Menší než	<code>a < b</code>
<code>></code>	Větší než	<code>a > b</code>
<code><=</code>	Menší nebo rovno	<code>a <= b</code>

Operátor	Význam	Příklad
<code>>=</code>	Větší nebo rovno	<code>a >= b</code>

Nejčastější chyba začátečníků: Záměna `=` (přiřazení) a `==` (porovnání). Podmínka `if (x = 5)` je v C# chybou kompilace, ale v jiných jazycích může tiše způsobit nesprávné chování. Vždy kontrolujte, že v podmínce používáte `==`.

Logické operátory

Operátor	Význam	Příklad
<code>&&</code>	Logické AND (a zároveň)	<code>a > 0 && a < 10</code>
<code>,</code>		<code>,</code>
<code>!</code>	Logické NOT (negace)	<code>!jeStudent</code>

```
int vek = 20;
bool jeZletily = vek >= 18;

bool muzeVolit = jeZletily && vek < 100; // 18 <= věk < 100
bool mimoPracovniVek = vek < 18 || vek >= 65; // pod 18 nebo nad 65
bool neniStudent = !jeStudent;
```

Zkrácené vyhodnocování (short-circuit evaluation):

- `A && B` — pokud je `A` nepravda, `B` se vůbec nevyhodnotí.
- `A || B` — pokud je `A` pravda, `B` se vůbec nevyhodnotí.

Toto je užitečné pro bezpečné operace:

```
// Kontrola dělitele před dělením:
if (b != 0 && a / b > 5)
{
    Console.WriteLine("Podíl je větší než 5.");
}
```

Priorita operátorů

Operátory mají různou prioritu (stejně jako v matematice násobení předchází sčítání). Od nejvyšší po nejnižší:

- `()` — závorky

2. `!`, `++`, `--`, `(int)`, `(double)` ... — negace, inkrementace, přetypování
3. `*`, `/`, `%` — násobení, dělení, modulo
4. `+`, `-` — sčítání, odčítání
5. `<`, `>`, `<=`, `>=` — relační
6. `==`, `!=` — rovnost
7. `&&` — logické AND
8. `||` — logické OR
9. `=`, `+=`, `-=` apod. — přiřazení

Doporučení: Pokud si nejste jisti prioritou, použijte závorky. Závorky kód zpřehledňují a zabrání chybám.

```
// Bez závorky: výsledek může být překvapivý
int vysledek = 2 + 3 * 4;      // 14 (ne 20)

// Se závorkou: záměr je jasný
int vysledek2 = (2 + 3) * 4;  // 20
```

Třída Math

Třída `Math` je součástí základní knihovny .NET a obsahuje sadu připravených matematických funkcí a konstant — odmocniny, mocniny, zaokrouhlování, absolutní hodnotu a další.

Nemusíte ji instalovat ani přidávat — je dostupná v každém C# programu.

```
Console.WriteLine(Math.Abs(-5));      // 5    - absolutní
hodnota
Console.WriteLine(Math.Sqrt(16));     // 4    - odmocnina
Console.WriteLine(Math.Pow(2, 10));   // 1024 - mocnina: 2^10
Console.WriteLine(Math.Round(3.567, 2)); // 3,57 - zaokrouhlení na 2
des. místa
Console.WriteLine(Math.Floor(3.9));   // 3    - zaokrouhlení
dolů
Console.WriteLine(Math.Ceiling(3.1)); // 4    - zaokrouhlení
nahoru
Console.WriteLine(Math.Max(10, 20));  // 20   - maximum
Console.WriteLine(Math.Min(10, 20));  // 10   - minimum
Console.WriteLine(Math.PI);           // 3,14159... Toto je
konstanta ne funkce :)
```

Pozor: `Math.Pow(8, 1/3)` vrátí `1,0`, nikoliv `2,0`. Proč? Výraz `1/3` je celočíselné dělení — výsledkem je `0`, takže se ve skutečnosti počítá `8^0 = 1`. Správně: `Math.Pow(8, 1.0/3)` nebo `Math.Pow(8, 1/3.0)`.

Shrnutí

- Celočíselné dělení `/` ořezává desetinnou část — pro desetinný výsledek musí být alespoň jeden operand desetinný.
- Operátor `%` vrací zbytek po dělení — užitečný pro test sudosti, cyklické indexování apod.
- `==` je porovnání, `=` je přiřazení — záměna je jednou z nejčastějších chyb.
- Pokud si nejste jisti prioritou, použijte závorky.

Kapitola 7: Ladění programu

Každý programátor dělá chyby — zkušení i začátečníci. Rozdíl je v tom, jak rychle je najdou a opraví. Tato kapitola vám ukáže, jaké typy chyb v programech vznikají a jak je efektivně hledat pomocí nástrojů, které Visual Studio nabízí.

Typy chyb

Typ chyby	Kdy se projeví	Příklad z praxe
Syntaktická (compile-time error)	Při překladu — program vůbec nespustí	Chybějící středník, závorka, překlep v názvu proměnné
Výjimka za běhu (runtime error)	Program se spustí, ale za běhu spadne	Uživatel zadá text místo čísla do <code>int.Parse()</code> , dělení nulou
Logická chyba (logic error)	Program běží, ale dává špatný výsledek	Špatný vzorec pro výpočet průměru, obrácené znaménko v podmínce

Nejobtížněji se hledají **logické chyby** — program se přeloží i spustí, ale počítá špatně. Kompilátor ani runtime o ničem neví, protože z jejich pohledu je vše v pořádku.

Kompilační chyby

Syntaktické chyby Visual Studio odhalí ještě předtím, než program spustíte — problematické místo podtrhne **červenou vlnovkou** přímo v editoru. Když na vlnovku najedete myší, zobrazí se stručný popis chyby.

Všechny chyby jsou také vidět v panelu **Error List** (zobrazit přes View → Error List). Každá chyba obsahuje:

- Popis chyby v angličtině.
- Číslo řádku, kde se chyba nachází.
- Kód chyby (například `CS0103`).

Poklepáním na chybu v Error Listu přejdete přímo na problematický řádek.

Rada: Opravujte chyby shora dolů. První chyba někdy způsobí, že kompilátor „ztratí orientaci“ a hlásí další falešné chyby níže, které zmizí samy po opravení té první.

Tip: Kód chyby (např. **CS0029**) můžete zkopírovat a vyhledat na webu — Microsoft má pro každý kód podrobnou dokumentaci s příklady.

Krokování (debugging)

Pokud program dává špatné výsledky a nestačí přechíst kód, použijte **debugger** — nástroj, který program zastaví na libovolném řádku a umožní vám sledovat, co se děje uvnitř.

Postup:

1. Klikněte do levého okraje editoru u řádku, kde chcete program zastavit — vytvoří se červený bod (**breakpoint**).
2. Spustíte program klávesou **F5** (ne Ctrl+F5 — to spustí program bez debuggeru).
3. Program se zastaví těsně před breakpointem — řádek se zvýrazní žlutě.
4. V panelu **Locals** (vlevo dole) sledujte aktuální hodnoty všech proměnných.
5. Postupujte řádek po řádku a sledujte, jak se hodnoty mění.

Klávesy pro krokování:

Klávesa	Akce
F10	Krok přes (Step Over) — provede aktuální řádek a zastaví na dalším
F11	Krok dovnitř (Step Into) — vstoupí do volané metody
Shift+F11	Krok ven (Step Out) — dokončí aktuální metodu a vrátí se
F5	Pokračovat do dalšího breakpointu
F9	Přidat/odebrat breakpoint na aktuálním řádku

Tip: V panelu Locals vidíte hodnoty proměnných v reálném čase. Pokud proměnná obsahuje jinou hodnotu, než jste čekali, jste blízko chybě. Pokud panel Locals nevidíte, zobrazte ho přes Debug → Windows → Locals (dostupné pouze během aktivního ladění, tedy po zastavení na breakpointu).

Příklad — program s logickou chybou:

Program má spočítat průměr tří čísel, ale dává špatný výsledek. Najdete chybu?

```
int a = 10;
int b = 20;
int c = 30;

double prumer = a + b + c / 3; // Výsledek bude 40, ne 20!

Console.WriteLine($"Průměr: {prumer}");
```

Chyba je v prioritě operátorů — dělení má vyšší prioritu než sčítání, takže se nejdřív vypočítá $c / 3 = 10$, a teprve pak $a + b + 10 = 40$. Závorky opraví pořadí výpočtu:

```
double prumer = (a + b + c) / 3.0; // Správně: 20
```

Zkuste si na tomto příkladu vyzkoušet breakpoint — zastavte program před výpočtem průměru a sledujte v Locals, jaká hodnota do proměnné `prumer` přibude.

Výpisy pro ladění

Nejjednodušší způsob ladění bez debuggeru — dočasně přidáte výpisy, které vám ukážou, co se v programu děje:

```
Console.WriteLine("Zadejte číslo: ");
int cislo = int.Parse(Console.ReadLine());

// Dočasný výpis pro ladění — po odladění odstraňte
Console.WriteLine($"[DEBUG] načtené číslo: {cislo}");

double vysledek = cislo * 1.21; // DPH 21 %

// Dočasný výpis pro ladění
Console.WriteLine($"[DEBUG] výsledek před zaokrouhlením: {vysledek}");

Console.WriteLine($"Cena s DPH: {Math.Round(vysledek, 2)}");
```

Prefix `[DEBUG]` pomáhá výpisy pro ladění snadno najít a po odladění odstranit.

Nejčastější logické chyby

```
// Špatné pořadí operandů při odčítání
double rozdil = b - a; // Chtěli jsme a - b

// Záměna priorit operátorů
double vysledek = a + b * c; // Nejdřív se násobí b * c, pak
// přičte a
double vysledek = (a + b) * c; // Se závorkami se nejdřív sečte a +
// b

// Celočíselné dělení místo desetinného
double prumer = (a + b) / 2; // ŠPATNĚ — výsledek je celé číslo
double prumer = (a + b) / 2.0; // SPRÁVNĚ

// Přetečení při výpočtu s velkými čísly
```

```
int velke = 1000000 * 1000000;    // ŠPATNĚ – přeteče int
long velke = 1000000L * 1000000L; // SPRÁVNĚ
```

Shrnutí

- Syntaktické chyby zachytí kompilátor — opravujte je shora dolů.
- Logické chyby se projeví špatným výsledkem — odhalíte je debuggerem nebo výpisem.
- Breakpointy a krokování jsou základní nástroje každého programátora.
- Testujte vždy s hodnotami, pro které znáte očekávaný výsledek.

Cvičení

Blok A z příkladů cvičení.

Kapitola 8: Větvení programu (if, switch)

Snippets ve Visual Studiu

Při psaní kódu nemusíte vždy psát celou strukturu ručně. Visual Studio nabízí **snippets** — připravené šablony kódu, které vložíte zkratkou: napište klíčové slovo a dvakrát stiskněte **Tab**. Od této chvíle vám snippets psaní kódu výrazně usnadní.

Snippet	Co vloží
<code>if</code> + Tab Tab	Prázdný blok <code>if</code>
<code>else</code> + Tab Tab	Blok <code>else</code>
<code>switch</code> + Tab Tab	Prázdný blok <code>switch</code> s jedním <code>case</code>

Po vložení snippetu jsou připravena místa pro doplnění — klávesou **Tab** přecházíte mezi nimi.

Příkaz if

Větvení umožňuje provádět různé části kódu podle toho, zda je splněna určitá podmínka.

Syntaxe:

```
if (podmínka)
{
    // Provede se, pokud podmínka je true
}
else
{

```

```
// Provede se, pokud podmínka je false (nepovinné)
}
```

Příklad:

```
Console.Write("Zadejte věk: ");
int vek = int.Parse(Console.ReadLine());

if (vek >= 18)
{
    Console.WriteLine("Jste zletilý/á, máte přístup.");
}
else
{
    Console.WriteLine("Jste nezletilý/á, přístup odepřen.");
}
```

Doporučení: V případě jednoho příkazu závorky `{}` povinné nejsou, ale doporučujeme je psát vždy. Zabrání to chybám při pozdějším přidávání řádků.

Větvení na více větví (else if)

```
Console.Write("Zadejte skóre (0-100): ");
int skore = int.Parse(Console.ReadLine());

if (skore >= 90)
{
    Console.WriteLine("Výborný (A)");
}
else if (skore >= 75)
{
    Console.WriteLine("Chvalitebný (B)");
}
else if (skore >= 60)
{
    Console.WriteLine("Dobrý (C)");
}
else if (skore >= 50)
{
    Console.WriteLine("Dostatečný (D)");
}
else
{
    Console.WriteLine("Nedostatečný (F)");
}
```

Rozbor příkladu:

- Podmínky se vyhodnocují shora dolů.
- Provede se **první** větev, jejíž podmínka je pravda.
- Ostatní větve se přeskočí.

Důležité: `else if` není totéž jako několik samostatných příkazů `if` za sebou. U řetězce `if / else if` se po splnění první podmínky ostatní větve přeskočí — provede se nejvýše jedna. U samostatných příkazů `if` se každý vyhodnotí zvlášť a může se provést více bloků najednou.

```
// Provede se jen jedna větev:
if (skore >= 90) { Console.WriteLine("A"); }
else if (skore >= 75) { Console.WriteLine("B"); } // přeskočí
se, pokud platilo první

// Provede se každá podmínka zvlášť – může se vypsát víc řádků:
if (skore >= 90) { Console.WriteLine("Výborný"); }
if (skore >= 75) { Console.WriteLine("Nad průměrem"); } //
může platit zároveň!
```

Vnořené příkazy if

```
bool maRidicak = true;
bool jeStrizlivy = true;

if (maRidicak)
{
    if (jeStrizlivy)
    {
        Console.WriteLine("Může řídit.");
    }
    else
    {
        Console.WriteLine("Má řidičák, ale nesmí řídit – není
strizlivý.");
    }
}
else
{
    Console.WriteLine("Nemá řidičský průkaz.");
}
```

Pozor na záludnou chybu — prázdný příkaz `if`:

```
// ŠPATNĚ – středník za if vytváří prázdný příkaz!
if (x == 0);           // Tato podmínka neřídí nic
    x = 5;             // Tento řádek se provede vždy!

// SPRÁVNĚ
if (x == 0)
{
    x = 5;
}
```

Pro zvědavé: ternární operátor

Zkrácená forma podmínky `if-else` na jednom řádku:

```
// Syntaxe: podmínka ? hodnotaJeliPravda : hodnotaJeliNepravda
int a = 10, b = 20;
int maximum = a > b ? a : b;
Console.WriteLine($"Maximum je: {maximum}"); // 20

string popis = vek >= 18 ? "zletilý" : "nezletilý";
```

Ternární operátor používejte pouze pro jednoduché podmínky. Pro složitější logiku je přehlednější klasický `if-else`.

Příkaz switch

Vhodný, pokud testujeme jednu proměnnou na více konkrétních hodnot.

```
Console.Write("Zadejte číslo dne (1-7): ");
int den = int.Parse(Console.ReadLine());

switch (den)
{
    case 1:
        Console.WriteLine("Pondělí");
        break;
    case 2:
        Console.WriteLine("Úterý");
        break;
    case 3:
        Console.WriteLine("Středa");
        break;
    case 4:
        Console.WriteLine("Čtvrtek");
        break;
```

```

    case 5:
        Console.WriteLine("Pátek");
        break;
    case 6:
    case 7:
        Console.WriteLine("Víkend");
        break;
    default:
        Console.WriteLine("Neplatné číslo dne.");
        break;
}

```

Pravidla příkazu switch:

- `case` definuje konkrétní hodnotu, na kterou se testuje.
- Každá větev musí být ukončena příkazem `break`.
- Dvě větve `case` bez příkazů mezi nimi (case 6, case 7) se chovají jako logické OR — provede se stejný blok.
- `default` je nepovinná větev, která se provede, pokud žádný `case` nesedí.
- V selektoru lze použít `int`, `char`, `string` nebo `enum`.

switch s řetězcem:

```

string mena = "USD";
double kurz;

switch (mena)
{
    case "USD":
        kurz = 23.5;
        break;
    case "EUR":
        kurz = 25.2;
        break;
    case "GBP":
        kurz = 29.1;
        break;
    default:
        Console.WriteLine($"Neznámá měna: {mena}");
        kurz = 0;
        break;
}

if (kurz > 0)
{
    Console.WriteLine($"100 {mena} = {100 * kurz:F0} Kč");
}

```

Pro zvědavé moderní switch výraz (C# 8+)

Novější syntaxe switch je kompaktnější:

```
int den = 3;
string nazevDne = den switch
{
    1 => "Pondělí",
    2 => "Úterý",
    3 => "Středa",
    4 => "Čtvrtek",
    5 => "Pátek",
    6 or 7 => "Víkend",
    _ => "Neplatné číslo"
};

Console.WriteLine(nazevDne);
```

Nejčastější chyby v podmínkách

```
// CHYBA: Záměna = a ==
if (x = 5) { ... } // Chyba kompilace v C# – naštěstí

// CHYBA: Porovnání desetinných čísel na přesnou rovnost
double d = 0.1 + 0.2;
if (d == 0.3) { ... } // Nemusí být pravda kvůli nepřesnosti float!
// SPRÁVNĚ: porovnávat s malou tolerancí (podrobněji probereme v
kurzu unit testování)
if (Math.Abs(d - 0.3) < 0.000001) { ... }

// CHYBA: Zapomenutý break ve switch
case 1:
    Console.WriteLine("Jedna");
    // Chybí break – kompilátor C# to jako chybu vyžaduje
```

Shrnutí

- `if` umožňuje podmíněné provádění bloků kódu.
- `else if` slouží pro testování více podmínek za sebou.
- `switch` je přehlednou alternativou, pokud testujeme jednu proměnnou na konkrétní hodnoty.
- Vždy používejte složené závorky `{}`, i pro jednořádkové bloky.

Cvičení

Blok B z příkladů cvičení.

Kapitola 9: Cykly (for, while, do-while)

Cykly umožňují opakovat blok příkazů — buď pevně daný počet krát, nebo dokud platí podmínka.

Cyklus for

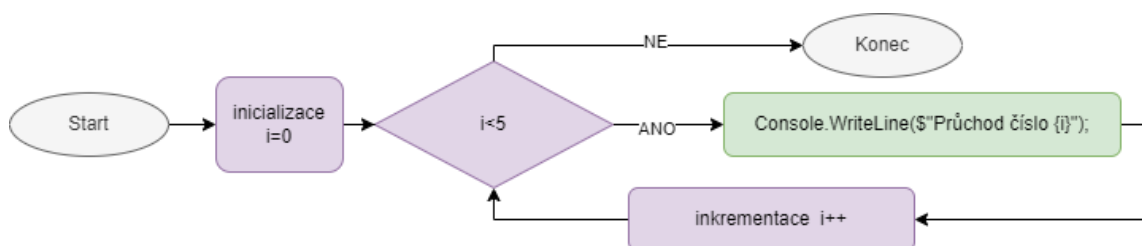
Vhodný, pokud **předem víme, kolikrát** má cyklus proběhnout.

```
// for (inicializace; podmínka; inkrementace)
for (int i = 0; i < 5; i++)
{
    Console.WriteLine($"Průchod číslo {i}");
}
```

Výstup:

```
Průchod číslo 0
Průchod číslo 1
Průchod číslo 2
Průchod číslo 3
Průchod číslo 4
```

Průběh cyklu for:



1. `int i = 0` — inicializace se provede jednou před začátkem cyklu.
2. `i < 5` — podmínka se vyhodnotí před každým průchodem. Pokud je `false`, cyklus končí.
3. Tělo cyklu se provede.
4. `i++` — inkrementace se provede na konci každého průchodu.
5. Přejde se zpět na krok 2.

Praktické příklady:

```
// Součet čísel od 1 do 100
int soucet = 0;
for (int i = 1; i <= 100; i++)
{
    soucet += i;
}
Console.WriteLine($"Součet 1 až 100 = {soucet}"); // 5050

// Výpis násobilky 7
for (int i = 1; i <= 10; i++)
{
    Console.WriteLine($"7 * {i} = {7 * i}");
}

// Cyklus se záporným krokem (sestupně)
for (int i = 10; i >= 1; i--)
{
    Console.Write($"{i} ");
}
Console.WriteLine(); // Odřádkování na konci
```

Doporučení: Jako proměnná cyklu se tradičně používá `i` (z anglického „index“). Pro vnořené cykly pak `j`, `k`.

Důležité: Nepoužívejte desetinné číslo jako řídicí proměnnou cyklu. Vnitřní reprezentace desetinných čísel způsobuje drobné odchylky, které mohou způsobit, že cyklus proběhne o jednu méně nebo více, než očekáváte.

```
// ŠPATNĚ – nepředvídatelný počet průchodů
for (double x = 0.1; x <= 0.3; x += 0.1)
{
    Console.WriteLine(x);
}

// SPRÁVNĚ – celočíselný index, dělení uvnitř
for (int i = 1; i <= 3; i++)
{
    Console.WriteLine(i / 10.0);
}
```

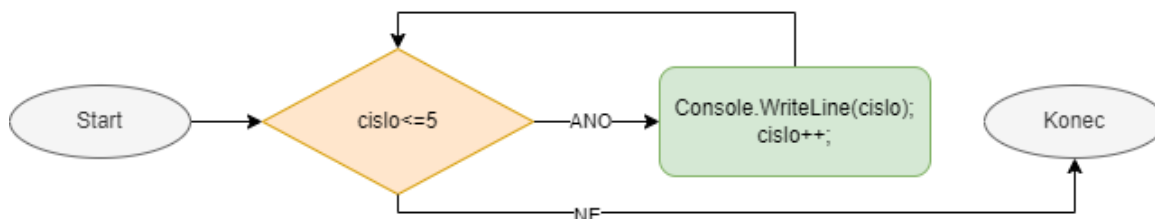
Cvičení

Blok C Jednoduché cykly (FOR) z příkladů cvičení.

Cyklus while

Vhodný, pokud počet průchodů předem **neznáme** — cyklus se opakuje, dokud platí podmínka.

Podmínka se vyhodnotí před každým průchodem. Pokud je hned na začátku nepravda, tělo cyklu se neprovede ani jednou.



Příklad: Odpočet — pokud uživatel zadá 0 nebo záporné číslo, cyklus vůbec neproběhne:

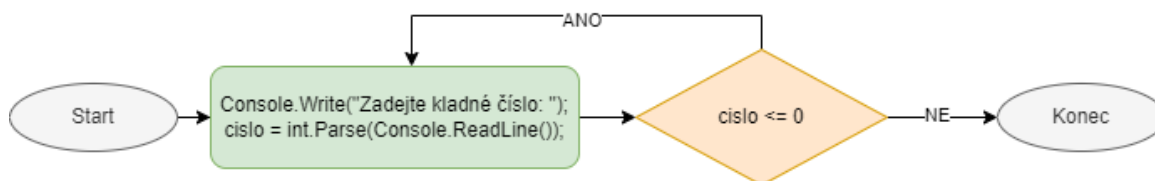
```

Console.Write("Zadejte počet sekund odpočtu: ");
int sekundy = int.Parse(Console.ReadLine());

while (sekundy > 0)
{
    Console.WriteLine($"Zbývá: {sekundy} s");
    sekundy--;
}
Console.WriteLine("Čas vypršel!");
  
```

Cyklus do-while

Podobný jako **while**, ale podmínka se vyhodnotí **na konci** — tělo cyklu se provede **vždy alespoň jednou**. Blok **do-while** lze rychle vložit snippetem: napište **do** a dvakrát stiskněte **Tab**.



Příklad: Zadání PIN — kód musíme načíst vždy alespoň jednou:

```

string spravnyPin = "1234";
string zadanyPin;

do
{
    Console.WriteLine("Zadejte PIN: ");
    zadanyPin = Console.ReadLine();
}
  
```

```

    if (zadanyPin != spravnyPin)
    {
        Console.WriteLine("Nesprávný PIN, zkuste znovu.");
    }
} while (zadanyPin != spravnyPin);

Console.WriteLine("Přístup povolen.");

```

Poznámka: PIN musíme načíst vždy alespoň jednou, proto se **do-while** hodí přirozeně. V praxi ale často fungují obě varianty — volba záleží na tom, jak přehledně dokážete podmínku vyjádřit.

Kdy použít while a kdy do-while? Pokud může nastat situace, kdy tělo cyklu nemá proběhnout ani jednou, použijte **while**. Pokud víte, že tělo musí proběhnout vždy alespoň jednou — například při prvním zobrazení menu nebo načtení vstupu — použijte **do-while**.

Cvičení

Blok D Jednoduché cykly While z příkladů cvičení.

Příkazy break a continue

break — okamžitě ukončí cyklus:

```

for (int i = 0; i < 100; i++)
{
    if (i == 5)
    {
        break; // Cyklus skončí při i == 5
    }
    Console.WriteLine(i);
}
// Vypíše: 0, 1, 2, 3, 4

```

continue — přeskočí zbytek aktuálního průchodu a pokračuje dalším:

```

for (int i = 0; i < 10; i++)
{
    if (i % 2 == 0)
    {
        continue; // Přeskočí sudá čísla
    }
    Console.WriteLine(i); // Vypíše jen lichá: 1, 3, 5, 7, 9
}

```

Doporučení pro `break` a `continue`: Oba příkazy používejte střídavě — kód s jejich přemírou se obtížně čte a sleduje. Zkuste nejprve vyjádřit logiku přímo v podmínce cyklu. `break` a `continue` jsou vhodné hlavně tehdy, když by alternativa byla výrazně složitější. Pozor: `break` ve vnořeném cyklu ukončí jen **vnitřní** cyklus, vnější pokračuje dál. U `continue` v cyklu `while` nebo `do-while` si ověřte, že se řídící proměnná aktualizuje před příkazem `continue` — jinak hrozí nekonečný cyklus.

Vnořené cykly

```

// Tisk násobilky (tabulka 10x10)
for (int i = 1; i <= 10; i++)
{
    for (int j = 1; j <= 10; j++)
    {
        Console.Write($"{i * j,4}"); // ,4 = zarovnání na 4 znaky
        // zprava – čísla jsou přehledně pod sebou
    }
    Console.WriteLine(); // Nový řádek po každé řadě
}

```

Výstup (první tři řádky):

```

1  2  3  4  5  6  7  8  9 10
2  4  6  8 10 12 14 16 18 20
3  6  9 12 15 18 21 24 27 30
...

```

Výkonnostní poznámka: Vnořené cykly jsou mocné, ale pozor na jejich složitost. Dva vnořené cykly od 0 do N mají složitost N^2 , tedy pro $N=1000$ proběhne tělo vnitřního cyklu milionkrát. Vždy se zeptejte, zda vnořený cyklus nelze nahradit efektivnějším algoritmem.

Nejčastější chyby v cyklech

```
// CHYBA: Nekonečný cyklus – zapomněli jsme přečíst vstup, heslo se
nikdy nezmění
int heslo = 0;
while (heslo != 1234)
{
    Console.WriteLine("Zadejte heslo:");
    // Zapomněli jsme: heslo = int.Parse(Console.ReadLine());
}

// CHYBA: Cyklus off-by-one – proběhne o jednu méně
for (int i = 1; i < 10; i++) // Proběhne 9x (1 až 9), ne 10x
{ ... }
// SPRÁVNĚ pro 10 průchodů:
for (int i = 0; i < 10; i++) // i = 0, 1, ..., 9
for (int i = 1; i <= 10; i++) // i = 1, 2, ..., 10

// CHYBA: Manipulace s řídicí proměnnou uvnitř cyklu
for (int i = 0; i < 10; i++)
{
    i += 2; // Vyhněte se – matoucí a chybové
}
```

Shrnutí

- **for** — vhodný pro pevný počet opakování.
- **while** — vhodný, pokud počet opakování závisí na podmínce.
- **do-while** — jako **while**, ale tělo proběhne vždy alespoň jednou.
- **break** ukončí cyklus, **continue** přeskočí aktuální průchod.

Cvičení

Blok C Dvojitý cyklus for z příkladů cvičení.

Kapitola 10: Pole

Než se pustíme do práce s polem, je dobré vědět jednu důležitou věc: pole je v C# **referenční typ**. To znamená, že proměnná pole neobsahuje přímo hodnoty, ale **odkaz (= referenci) na místo v operační paměti**, kde jsou hodnoty skutečně uloženy.

V praxi to má jeden důležitý důsledek — přiřazení pole do jiné proměnné nevytvoří kopii, jen druhý odkaz na stejná data. Tohle chování uvidíme konkrétně v sekci [Kopírování pole](#).

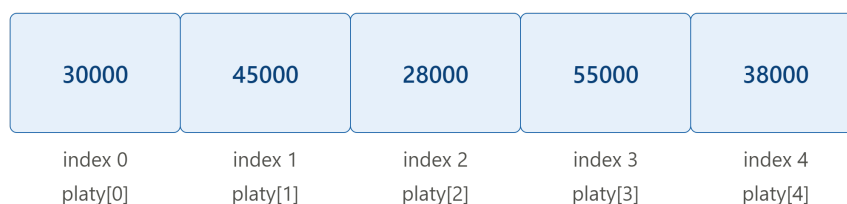
Naopak jednoduché typy jako **int**, **double** nebo **bool** jsou **hodnotové typy** — přiřazení vždy vytvoří nezávislou kopii. Hodnotové typy jsou uloženy na **zásobníku** (stack), zatímco pole a ostatní referenční typy jsou uloženy na **haldě** (heap), se kterou se pracuje trochu jinak. S tím vším se podrobněji setkáte v kurzu OOP.

Co je pole

Pole je kolekce prvků stejného datového typu uložených za sebou v paměti. Každý prvek má přiřazen číselný **index** začínající od **0**. Pole si můžete představit např. jako bytovku - byt číslo 1 obývají Novákové, byt číslo 2 paní Svobodová a tak, případně jako poličku s kořenkami - zleva máte skořici, pepř, kmín... správnou kořenku najdete buď podle štítku, nebo i poslepu, pokud vám je samozřejmě někdo nezpřehází.

```
// Pole 5 celých čísel – indexy 0 až 4
int[] platy = new int[5];
platy[0] = 30000;
platy[1] = 45000;
platy[2] = 28000;
platy[3] = 55000;
platy[4] = 38000;
```

int[] platy



Typová bezpečnost: Pole přijme pouze hodnoty svého datového typu — do `int[]` nelze vložit text, do `string[]` číslo. Kompilátor takový pokus rovnou odmítne. Zkuste si to!

Inicializace pole

Na rozdíl od jednoduchých proměnných (`int i = 3`) pole není jedna hodnota — je to **rezervace místa v paměti pro více hodnot najednou**. Proto musíte říct, kolik prvků má pole obsahovat, a teprve potom prvky plnit.

```
// Způsob 1: Nejprve rezervovat místo, pak postupně plnit
int[] cisla = new int[3]; // Vytvoří pole pro 3 čísla (zatím
// všechna jsou 0)
cisla[0] = 10;
cisla[1] = 20;
cisla[2] = 30;

// Způsob 2: Inicializace při deklaraci – velikost se odvodí
// automaticky
```

```
int[] hodnoceni = { 5, 4, 3, 4, 5, 3 };

// Způsob 3: Kombinace – explicitní typ i hodnoty
string[] dny = new string[] { "Po", "Út", "St", "Čt", "Pá", "So",
    "Ne" };
```

Výchozí hodnoty: Pokud vytvoříte pole bez přiřazení hodnot (`new int[3]`), C# automaticky doplní výchozí hodnoty — pro `int` je to `0`, pro `bool` je to `false`, pro `string` je to `null`.

Přístup k prvkům a délka pole

K jednotlivým prvkům pole se přistupuje pomocí **indexu v hranatých závorkách**. Index říká, na které místo v poli sáhnout — první místo má vždy index `0`.

```
int[] cisla = { 10, 20, 30, 40, 50 };

Console.WriteLine(cisla[0]);           // 10 – první prvek
Console.WriteLine(cisla[4]);           // 50 – poslední prvek
Console.WriteLine(cisla.Length);       // 5 – počet prvků

// Změna hodnoty prvku
cisla[2] = 99;
Console.WriteLine(cisla[2]);           // 99

// Poslední prvek – bezpečný způsob bez nutnosti znát délku
Console.WriteLine(cisla[cisla.Length - 1]); // 50
```

Proč začíná index od 0? Jde o konvenci zděděnou z nižších úrovní programování, kde index vyjadřuje *offset od začátku pole v paměti*. První prvek je hned na začátku — tedy offset `0`. Dnes je to prostě pravidlo, které si zapamatujete po prvním `IndexOutOfRangeException`.

Nejčastější chyba: Pole s 5 prvky má platné indexy `0` až `4`. Přístup na index `5` (nebo vyšší) způsobí výjimku `IndexOutOfRangeException`. Pozor zejména při psaní podmínek v cyklu — viz sekce [Procházení pole cyklem](#).

Procházení pole cyklem

Protože víme, kolik prvků pole má (vlastnost `Length`), můžeme ho projít cyklem — přistoupit postupně na každý index od `0` do `Length - 1`.

```
int[] teploty = { 18, 22, 25, 19, 21 };

// Způsob 1: for s indexem
```



```

for (int i = 0; i < teploty.Length; i++)
{
    Console.WriteLine($"Den {i + 1}: {teploty[i]}°C");
}

// Způsob 2: foreach (čtení bez modifikace)
foreach (int t in teploty)
{
    Console.Write($"{t} ");
}
Console.WriteLine();

```

Proč `i < Length`, a ne `i <= Length`? Pole s 5 prvky má `Length = 5`, ale indexy jsou 0, 1, 2, 3, 4 — poslední platný index je tedy 4, neboli `Length - 1`. Podmínka `i < 5` zastaví cyklus ve chvíli, kdy `i` dosáhne hodnoty 5 — cyklus tedy skončí na indexu 4, což je správně. Kdybychom napsali `i <= teploty.Length`, cyklus by se pokusil přistoupit na index 5, který neexistuje, a program by spadl s výjimkou `IndexOutOfRangeException`.

for vs foreach: `foreach` je čitelnější pro prosté procházení, kdy nepotřebujeme vědět, na kterém indexu jsme. Pokud potřebujete index (např. vypsát "Den 1, Den 2...") nebo chcete prvky měnit, použijte `for`.

Praktický příklad: Statistiky

```

namespace StatistikaPole;

internal class Program
{
    static void Main(string[] args)
    {
        int[] znamky = { 2, 1, 3, 2, 1, 4, 2, 3, 1, 2 };

        // Součet a průměr
        int soucet = 0;
        foreach (int z in znamky)
        {
            soucet += z;
        }
        double prumer = (double)soucet / znamky.Length;

        // Maximum – inicializujeme prvním prvkem, ne nulou!
        int max = znamky[0];
        for (int i = 1; i < znamky.Length; i++)
        {
            if (znamky[i] > max)
            {
                max = znamky[i];
            }
        }
    }
}

```

```

    }

    // Minimum
    int min = znamky[0];
    for (int i = 1; i < znamky.Length; i++)
    {
        if (znamky[i] < min)
        {
            min = znamky[i];
        }
    }

    Console.WriteLine($"Počet: {znamky.Length}");
    Console.WriteLine($"Průměr: {prumer:F2}");
    Console.WriteLine($"Maximum: {max}");
    Console.WriteLine($"Minimum: {min}");
}
}

```

Proč inicializovat max na `znamky[0]` a ne na `0`? Pokud by všechny hodnoty byly záporné, inicializace na `0` by jako maximum vrátila `0`, které v poli vůbec není. Správně se inicializuje na první prvek pole.

Kopírování pole

Pole v C# je **referenční typ** — proměnná neobsahuje přímo hodnoty, ale odkaz na místo v paměti, kde jsou hodnoty uloženy. Přiřazení `kopie = original` proto nevytvoří nové pole, jen druhý odkaz na to stejné. Změna vytvořená přes proměnnou `kopie` pak ovlivní i `original`.

```

int[] original = { 1, 2, 3, 4, 5 };

// ŠPATNĚ – nevznikne kopie, jen druhý odkaz na stejné pole!
int[] spatnaKopie = original;
spatnaKopie[0] = 99;
Console.WriteLine(original[0]); // 99 – original byl změněn!

// SPRÁVNĚ – skutečná kopie
int[] spravnaKopie = original.ToArray();
spravnaKopie[0] = 99;
Console.WriteLine(original[0]); // 1 – original nebyl změněn

```

Toto chování se týká všech referenčních typů v C# — tedy nejen polí. U jednoduchých proměnných (`int`, `double`, `bool`) k tomu nedochází, protože ty jsou hodnotové typy a přiřazení vždy vytvoří kopii. Více o referenčních a hodnotových typech se dozvíte v kurzu OOP.

Dvourozměrná pole

```
// Matice 3 řádky × 4 sloupce
int[,] matice = new int[3, 4];

// Inicializace
int[,] sudoku = {
    { 5, 3, 4, 6 },
    { 6, 7, 2, 1 },
    { 1, 9, 8, 3 }
};

// Přístup: matice[řádek, sloupec]
Console.WriteLine(sudoku[0, 0]); // 5 – řádek 0, sloupec 0
Console.WriteLine(sudoku[1, 2]); // 2 – řádek 1, sloupec 2

// Rozměry
Console.WriteLine(sudoku.GetLength(0)); // 3 – počet řádků
Console.WriteLine(sudoku.GetLength(1)); // 4 – počet sloupců

// Procházení dvourozměrného pole
for (int r = 0; r < sudoku.GetLength(0); r++)
{
    for (int s = 0; s < sudoku.GetLength(1); s++)
    {
        Console.Write($"{sudoku[r, s]} ");
    }
    Console.WriteLine();
}
```

Třída Array a vlastnosti pole

V C# je každé pole ve skutečnosti instancí třídy **Array**. To znamená, že pole automaticky získá řadu užitečných vlastností a metod — část z nich voláme přímo na proměnné pole (tečkovou notací), část přes statickou třídu **Array**.

Vlastnosti a metody volané přímo na poli:

```
int[] cisla = { 5, 2, 8, 1, 9, 3 };

Console.WriteLine(cisla.Length); // 6 – celkový počet prvků
Console.WriteLine(cisla.GetLength(0)); // 6 – počet prvků v rozměru 0
// u jednorozměrného pole = totéž jako Length

int[,] matice = new int[3, 4];
```

```

Console.WriteLine(matice.GetLength(0)); // 3 – počet řádků
Console.WriteLine(matice.GetLength(1)); // 4 – počet sloupců
                                     // u dvourozměrného pole

Length nestačí

```

Metody třídy Array — voláme jako `Array.NázevMetody(pole)`:

```

int[] ciska = { 5, 2, 8, 1, 9, 3 };

Array.Sort(ciska); // Seřazení
vzestupně
Console.WriteLine(string.Join(", ", ciska)); // 1, 2, 3, 5, 8, 9

Array.Reverse(ciska); // Obrácení pořadí
Console.WriteLine(string.Join(", ", ciska)); // 9, 8, 5, 3, 2, 1

int index = Array.IndexOf(ciska, 5); // Hledání hodnoty
– vrátí index
Console.WriteLine(index); // 2

```

Generování náhodných čísel

Třída `Random` slouží k vytváření pseudonáhodných čísel — čísel, která vypadají náhodně, ale jsou generována **deterministickým algoritmem**. Deterministický znamená, že výsledek je vždy jednoznačně určen vstupními daty — algoritmus nezahrnuje žádnou skutečnou náhodu. Pro běžné použití (hry, simulace, testovací data) jsou taková čísla zcela dostačující.

Nejprve vytvoříme instanci generátoru, pak z něj čísla čerpáme:

```

Random rng = new Random();

// Celé číslo od 1 do 6 (včetně) – simulace hodu kostkou
int kostka = rng.Next(1, 7); // Next(min, max) – max je VÝLUČNÉ!
Console.WriteLine(kostka);

// Celé číslo od 0 do 99
int nahodne = rng.Next(0, 100);

// Desetinné číslo od 0.0 do 1.0 (včetně 0, výlučně 1)
double desetinne = rng.NextDouble();

// Naplnění pole náhodnými čísly od 0 do 99
int[] nahodna = new int[10];
for (int i = 0; i < nahodna.Length; i++)
{

```

```
nahodna[i] = rng.Next(0, 100);
}
```

Pozor na `Next(min, max)`: Horní hranice je vždy **výlučná** — výsledek ji nikdy nedosáhne. V matematice se takový rozsah zapisuje jako otevřený interval `[1, 7)`: levá hranice je uzavřená (špičatá závorka — číslo 1 může padnout), pravá je otevřená (kulatá závorka — číslo 7 nikdy nepadne). Proto pro čísla 1–6 píšeme `Next(1, 7)`, ne `Next(1, 6)`.

Jeden generátor pro celý program: Vytvářejte instanci `Random` jen jednou a používejte ji opakovaně. Vytváření nového `Random` v každém volání může vést k opakování stejných hodnot.

Shrnutí

- Pole ukládá více hodnot stejného typu pod jedním názvem.
- Indexování začíná od 0, poslední index je `Length - 1`.
- Pro procházení bez modifikace použijte `foreach`, s indexem nebo modifikací `for`.
- Přiřazení pole nevytváří kopii — pro kopii použijte `.ToArray()`.
- Maximum a minimum inicializujte vždy do první prvek pole, nikoli na nulu.

Cvičení

Blok E Cykly a pole z příkladů cvičení.

Kapitola 11: Funkce (metody)

Proč funkce

V mnoha programovacích jazycích se pojmenovaným blokům kódu říká **funkce**. V C# se jim říká **metody** — proto, že v C# každý kód patří do třídy, a funkce, která patří do třídy, se nazývá metoda. Jde tedy o totéž, jen v kontextu objektově orientovaného programování. S třídami se podrobněji seznámíte v kurzu OOP, zatím stačí vědět, že v C# správně říkáme metoda.

Metody slouží ke:

- **Rozdělení programu** na menší, přehledné celky.
- **Opětovnému použití kódu** — jednou napíšete, voláte vícekrát.
- **Snadnější údržbě** — oprava chyby na jednom místě se projeví všude.

Pro zvědavé: V různých jazycích a prostředích se podobné konstrukce nazývají různě — **podprogram**, **procedura**, **subrutina** nebo **subprogram**. Procedura obvykle označuje blok kódu bez návratové hodnoty (v C# odpovídá metodě s `void`), funkce pak blok s návratovou hodnotou. V C# se pro oboje používá jednotný pojem **metoda**.

Definice a volání metody

Metoda se skládá ze dvou kroků: nejprve ji **definujeme** (říkáme, co má dělat), pak ji **voláme** (říkáme, kdy to má udělat).

```
// Definice metody – říkáme, jak se metoda jmenuje, co dostane a co vrátí
static int Secti(int a, int b)
{
    return a + b;
}

// Volání metody – předáme konkrétní hodnoty, výsledek uložíme do proměnné
int vysledek = Secti(3, 7);
Console.WriteLine(vysledek); // 10
```

Syntaxe metody — každá část má svůj význam:

```
static návratovýTyp NázevMetody(typ1 parametr1, typ2 parametr2)
{
    // Tělo metody – co metoda provede
    return hodnota; // Povinné, pokud návratový typ není void
}
```

- **static** — prozatím píšeme u všech metod; souvisí s třídami, více v kurzu OOP.
- **návratový typ** — datový typ hodnoty, kterou metoda vrátí (**int**, **double**, **string**, ...). Pokud metoda nic nevrací, píšeme **void**.
- **název metody** — pojmenováváte výstižně podle toho, co metoda dělá. Konvence je **PascalCase**: **VypoctiPrumer**, **ZjistitMaximum**.
- **parametry** — vstupy metody; každý má svůj typ a název. Metoda může být i bez parametrů — závorky zůstanou prázdné.
- **return** — ukončí metodu a vrátí hodnotu volajícímu kódu. U **void** metod není povinný — pokud ho použijete, musí být bez hodnoty a slouží k předčasnému ukončení metody.

Metoda bez návratové hodnoty (void)

void metoda nic nevrací — místo výsledku provádí **akci**: vypíše text na obrazovku, uloží data do souboru, změní stav programu. Zajímá nás, že se něco stane, ne že dostaneme zpět nějakou hodnotu. Ostatně **Console.WriteLine**, které používáte od první kapitoly, je také **void** metoda.

```
static void VypisPozdrav(string jmeno)
{
```

```

        Console.WriteLine($"Ahoj, {jmeno}!");
        Console.WriteLine("Vítejte v programu.");
    }

    // Volání – výsledek nikam neukládáme, metoda jen provede akci
    VypisPozdrav("Jana");
    VypisPozdrav("Petr");

```

Metoda vracející hodnotu

Pokud metoda něco zjistí nebo zpracuje a my chceme výsledek použít dál v programu, metoda hodnotu **vrátí** příkazem **return**. Návrátový typ může být jakýkoliv — číslo, text, logická hodnota i jiné typy. Kompilátor hlídá, aby návratový typ v definici metody odpovídal tomu, co **return** skutečně vrátí.

```

// Vrací číslo – výsledek výpočtu
static double VypoctiObsahKruhu(double polomer)
{
    return Math.PI * polomer * polomer;
}

// Vrací bool – výsledek porovnání
static bool JePlnoletý(int vek)
{
    return vek >= 18;
}

// Vrací string – zpracovaný text
static string VytvorPozdrav(string jmeno)
{
    return $"Ahoj, {jmeno}!";
}

```

Vracenou hodnotu pak můžeme uložit do proměnné, nebo ji rovnou použít:

```

double obsah = VypoctiObsahKruhu(5.0);
Console.WriteLine($"Obsah kruhu s poloměrem 5 je: {obsah:F2}");

// Nebo bez uložení – rovnou v podmínce
if (JePlnoletý(20))
{
    Console.WriteLine("Vstup povolen.");
}

```

return ukončí metodu okamžitě — kód za ním se již neprovede. Metoda může mít více příkazů **return** v různých větvích **if**, ale vždy se provede jen jeden.

Praktický příklad: Trojúhelník

```
namespace Trojuhelnik;

internal class Program
{
    // Metoda vrací true, pokud zadané strany tvoří trojúhelník
    static bool JeTrojuhelnik(double a, double b, double c)
    {
        return a + b > c && a + c > b && b + c > a;
    }

    // Metoda vrací obvod trojúhelníku
    static double Obvod(double a, double b, double c)
    {
        return a + b + c;
    }

    // Metoda vrací obsah trojúhelníku (Heronův vzorec)
    static double Obsah(double a, double b, double c)
    {
        double s = Obvod(a, b, c) / 2;
        return Math.Sqrt(s * (s - a) * (s - b) * (s - c));
    }

    static void Main(string[] args)
    {
        Console.Write("Strana a: ");
        double a = double.Parse(Console.ReadLine());
        Console.Write("Strana b: ");
        double b = double.Parse(Console.ReadLine());
        Console.Write("Strana c: ");
        double c = double.Parse(Console.ReadLine());

        if (JeTrojuhelnik(a, b, c))
        {
            Console.WriteLine($"Obvod: {Obvod(a, b, c):F2}");
            Console.WriteLine($"Obsah: {Obsah(a, b, c):F2}");
        }
        else
        {
            Console.WriteLine("Zadané strany netvoří trojúhelník.");
        }
    }
}
```


Předávání parametrů hodnotou a odkazem

Standardně C# předává parametry **hodnotou** — metoda dostane kopii proměnné. Změna uvnitř metody neovlivní původní proměnnou.

```
static void Zvys010(int x)
{
    x += 10; // Mění jen lokální kopii
}

int cislo = 5;
Zvys010(cislo);
Console.WriteLine(cislo); // Stále 5 – metoda neovlivnila originál
```

V praxi se nejčastěji setkáte s klíčovým slovem **out**, které slouží k předání **výstupního parametru** — metoda do něj запиše výsledek. Proměnná nemusí být před voláním inicializována, ale metoda ji musí přiřadit. Typický příklad je **TryParse**, kterou budeme používat v kurzu OOP:

```
string vstup = Console.ReadLine();

if (int.TryParse(vstup, out int cislo))
{
    Console.WriteLine($"Zadali jste číslo: {cislo}");
}
else
{
    Console.WriteLine("To není číslo.");
}
```

TryParse vrátí **true** nebo **false** podle toho, zda převod proběhl úspěšně, a výsledek převodu předá přes **out** parametr **cislo**.

Pro zvědavé: Klíčové slovo **ref** umožňuje předat proměnnou skutečně odkazem — metoda pak pracuje přímo s originálem, ne s kopií. Na rozdíl od **out** musí být proměnná před voláním inicializována a metoda ji přiřadit nemusí. Více v kurzu OOP.

Pro zvědavé — přetěžování metod (method overloading): V C# může existovat více metod se stejným názvem, ale různými parametry. Kompilátor sám vybere správnou variantu podle předaných argumentů. Například **Console.WriteLine** funguje pro **int**, **double** i **string** právě díky přetěžování.

Pro zvědavé — rekurze: Metoda může volat sama sebe — tomu se říká rekurze. Každá rekurzivní metoda musí mít základní případ, který rekurzi ukončí. Bez něj nastane

nekonečná rekurze a program spadne s `StackOverflowException`. Typický učebnicový příklad je výpočet faktoriálu — $5! = 5 \times 4 \times 3 \times 2 \times 1$:

```
static long Faktorial(int n)
{
    if (n <= 1)
    {
        return 1; // Základní případ –
        // ukončuje rekurzi
    }
    return n * Faktorial(n - 1); // Rekurzivní volání
}

Console.WriteLine(Faktorial(5)); // 120
Console.WriteLine(Faktorial(10)); // 3628800
```

V praxi se rekurze hodí například pro procházení stromové struktury složek na disku, kde každá složka může obsahovat další složky.

Pole jako vstupní parametr

V kapitole o polích jsme výpočty jako součet nebo průměr psali přímo do `Main`. Teď je přesuneme do metod — kód bude přehlednější a výpočty půjde snadno použít vícekrát.

Metoda může přijímat pole jako parametr — v závorce se uvede typ s prázdnými hranatými závorkami. Velikost pole se v závorce neuvádí, aby metoda nebyla omezena na konkrétní délku.

```
static int Soucet(int[] pole)
{
    int s = 0;
    foreach (int hodnota in pole)
    {
        s += hodnota;
    }
    return s;
}

static void Main(string[] args)
{
    int[] cisla = { 10, 20, 30, 40 };
    Console.WriteLine($"Součet: {Soucet(cisla)}"); // 100
}
```

Na rozdíl od běžných proměnných hodnotových typů (`int`, `double`, ...) se pole předává **odkazem** — důvod jste už viděli v kapitole o polích: pole je referenční typ, takže proměnná obsahuje odkaz na místo v paměti, ne přímo data. Metoda proto pracuje přímo s originálním polem a změny uvnitř metody se projeví i ve volajícím kódu:

```
static void ZdvojHodnoty(int[] pole)
{
    for (int i = 0; i < pole.Length; i++)
    {
        pole[i] *= 2;
    }
}

static void Main(string[] args)
{
    int[] cisla = { 1, 2, 3 };
    ZdvojHodnoty(cisla);
    Console.WriteLine(string.Join(", ", cisla)); // 2, 4, 6 –
    // string.Join
    // spojí prvky pole do řetězce odděleného zadaným oddělovačem
}
```

Pozor: Toto chování je odlišné od předávání proměnných hodnotových typů. Pokud metoda nemá pole měnit, předejte ji kopii: `int[] kopie = pole.ToArray();`

Metoda vracející pole

Metoda může také vrátit pole — návratový typ se zapíše s prázdnými hranatými závorkami. Pole se uvnitř metody vytvoří příkazem `new` a vrátí příkazem `return`:

```
static double[] PripoctiUrok(double[] uspor, double urokSazba)
{
    double[] vysledek = new double[uspor.Length];
    for (int i = 0; i < uspor.Length; i++)
    {
        vysledek[i] = uspor[i] * (1 + urokSazba / 100);
    }
    return vysledek;
}

static void Main(string[] args)
{
    double[] vklady = { 10000, 25000, 5000 };
    double[] poUroceni = PripoctiUrok(vklady, 3.5);

    for (int i = 0; i < poUroceni.Length; i++)
```

```

    {
        Console.WriteLine($"Vklad {i + 1}: {poUroceni[i]:F2} Kč");
    }
}

```

Parametr `urokSazba` se předává v procentech (např. `3.5` znamená 3,5 %). Převod na desetinné číslo provádí vzorec `urokSazba / 100` uvnitř metody.

Volající kód pole přijme do proměnné stejného typu:

```
int[] vysledek = NazevMetody(parametry);
```

Poznámka: Metoda vrátí nové pole — originál předaný jako parametr zůstane nezměněný. Takto lze bezpečně transformovat data bez vedlejších efektů.

Praktický příklad: Výběr sudých čísel

```

static int[] VyberSuda(int[] pole)
{
    // Nejprve spočítáme, kolik sudých čísel pole obsahuje
    // (musíme znát velikost výsledného pole předem)
    int pocet = 0;
    foreach (int x in pole)
    {
        if (x % 2 == 0)
        {
            pocet++;
        }
    }

    // Vytvoříme výsledné pole správné velikosti
    int[] suda = new int[pocet];

    // Naplníme ho sudými čísly
    int index = 0;
    foreach (int x in pole)
    {
        if (x % 2 == 0)
        {
            suda[index] = x;
            index++;
        }
    }

    return suda;
}

```

```
static void Main(string[] args)
{
    int[] cisla = { 3, 8, 1, 6, 10, 7, 4 };
    int[] suda = VyberSuda(cisla);
    Console.WriteLine(string.Join(", ", suda)); // 8, 6, 10, 4
}
```

Viditelnost proměnných (rozsah platnosti)

Každá proměnná je platná (viditelná) jen v té části kódu, kde byla deklarována — tomu říkáme **rozsah platnosti** (anglicky *scope*). Překročit tento rozsah kompilátor nedovolí.

Lokální proměnné jsou deklarovány uvnitř metody nebo bloku `{ }`. Existují jen po dobu, kdy se daný blok vykonává, a po jeho skončení zanikají.

```
static void Main(string[] args)
{
    int x = 10; // Lokální proměnná metody Main

    for (int i = 0; i < 3; i++)
    {
        int y = i * 2; // Lokální proměnná platná jen uvnitř cyklu
        Console.WriteLine(y);
    }

    // Console.WriteLine(i); // Chyba! 'i' zde neexistuje –
    // skončila se smyčka
    // Console.WriteLine(y); // Chyba! 'y' zde neexistuje
    Console.WriteLine(x);    // OK – x je deklarováno v Main
}
```

Proměnná deklarovaná v hlavičce `for (int i)` je viditelná jen uvnitř těla toho cyklu. Toto chování je záměrné — zabráňuje nechtěnému použití proměnné tam, kde již nemá smysl.

Proměnné různých metod jsou na sobě zcela nezávislé. Pokud mají dvě metody proměnnou se stejným názvem, nejde o tutéž proměnnou — každá existuje jen ve své metodě.

```
static void MetodaA()
{
    int vysledek = 10;
    Console.WriteLine(vysledek); // 10
}

static void MetodaB()
```

```
{
    int vysledek = 99;           // Jiná proměnná, jen stejný název
    Console.WriteLine(vysledek); // 99
}
```

Globální proměnné

Proměnnou lze deklarovat i mimo jakoukoliv metodu — přímo ve třídě. Taková proměnná je **globální** a je přístupná ze všech metod třídy.

```
internal class Program
{
    static int pocitadlo = 0; // Globální proměnná – viditelná ve
    všech metodách

    static void Zvys()
    {
        pocitadlo++;
    }

    static void Main(string[] args)
    {
        Zvys();
        Zvys();
        Console.WriteLine(pocitadlo); // 2
    }
}
```

Doporučení: Globální proměnné používejte co nejméně. Metoda, která pracuje s globální proměnnou, je obtížněji přenositelná a testovatelná — nikde v jejím záhlaví nevidíte, co používá. Veškeré vstupy a výstupy metody předávejte přes parametry a návratové hodnoty.

Nejčastější chyby při práci s metodami

```
// CHYBA: Chybějící return ve větvi if
static float Podil(float x, float y)
{
    if (y != 0)
    {
        return x / y;
    }
    // Chyba kompilace: ne všechny cesty vrací hodnotu!
    // OPRAVA: přidat else s return nebo return za if blokem
}
```

```
// SPRÁVNĚ:
static float Podil(float x, float y)
{
    if (y == 0)
    {
        Console.WriteLine("Dělení nulou!");
        return 0;
    }
    return x / y;
}
```

Shrnutí

- Metody rozdělují program na přehledné celky a umožňují opakované použití kódu.
- `void` metoda nic nevrací, ostatní vrací hodnotu příkazem `return`.
- Parametry jsou standardně předávány hodnotou (kopii). Klíčové slovo `out` slouží pro výstupní parametry — metoda do nich zapíše výsledek.
- Pole jako parametr se předává odkazem na obsah — změny uvnitř metody mění originální pole.
- Metoda může vrátit pole — vytvoří ho uvnitř příkazem `new` a vrátí příkazem `return`.

Cvičení

Blok F Funkce bez pole z příkladů cvičení.

Blok F2 Funkce s polem z příkladů cvičení.

Kapitola 12: Práce s řetězci

Základní operace s řetězci

`string` v C# je **immutabilní** (neměnný) — jakmile je řetězec vytvořen, jeho obsah nelze změnit. Každá „úprava“ řetězce ve skutečnosti vytvoří nový řetězec v paměti, původní zůstane nezměněn. To má jeden důležitý praktický důsledek: výsledek metod jako `ToUpper()` nebo `Replace()` musíte vždy uložit do proměnné — volání samo o sobě původní řetězec nezmění.

```
string s = "Ahoj, světe!";

Console.WriteLine(s.Length);           // 12 – počet znaků
Console.WriteLine(s.ToUpper());        // AHOJ, SVĚTE!
Console.WriteLine(s.ToLower());        // ahoj, světe!
Console.WriteLine(s);                  // Ahoj, světe! – původní s
se nezměnil!

// Výsledek musíme uložit
```

```
string velka = s.ToUpper();
Console.WriteLine(velka);           // AHOJ, SVĚTE!
```

Spojování řetězců

Řetězce lze spojovat operátorem `+` (konkatenace) nebo interpolací `$""`, kterou už znáte:

```
string jmeno = "Jana";
string prijmeni = "Nováková";

// Konkatenace
string celeJmeno = jmeno + " " + prijmeni;
Console.WriteLine(celeJmeno);           // Jana Nováková

// Interpolace – čitelnější zápis
Console.WriteLine($"Jméno: {jmeno} {prijmeni}");
```

Nejpoužívanější metody třídy string

Změna velikosti písmen:

```
string s = "Ahoj, světe!";
Console.WriteLine(s.ToUpper());        // AHOJ, SVĚTE! – vše velkými
Console.WriteLine(s.ToLower());        // ahoj, světe! – vše malými
```

Ořezání bílých znaků — hodí se při zpracování vstupu od uživatele, který mohl omylem přidat mezery:

```
string text = " Hello, World! ";
Console.WriteLine(text.Trim());         // "Hello, World!" – ořeže z
obou stran
Console.WriteLine(text.TrimStart());    // "Hello, World! " – ořeže
zleva
Console.WriteLine(text.TrimEnd());      // " Hello, World!" – ořeže
zprava
```

Hledání v řetězci:

```
string s = "Programování v C#";
Console.WriteLine(s.Contains("C#"));    // True – obsahuje
podřetězec?
Console.WriteLine(s.StartsWith("Pro")); // True – začíná tímto
```



```

textem?
Console.WriteLine(s.EndsWith("C#"));           // True – končí tímto
textem?
Console.WriteLine(s.IndexOf("v"));             // 14 – vrátí index
prvního výskytu, nebo -1 pokud nenajde

```

Nahrazení — vrátí nový řetězec, kde jsou všechny výskyty hledaného textu nahrazeny:

```

string s = "Programování v C#";
Console.WriteLine(s.Replace("C#", "Javě")); // "Programování v
Javě"

```

Rozdělení — rozdělí řetězec podle oddělovače a vrátí pole řetězců:

```

string csv = "Jan,Novák,Praha,30";
string[] casti = csv.Split(',');
foreach (string cast in casti)
{
    Console.WriteLine(cast); // Jan / Novák / Praha / 30
}

```

Podřetězec — vrátí část řetězce od zadaného indexu:

```

string email = "jan.novak@example.com";
Console.WriteLine(email.Substring(0, 9)); // "jan.novak" – od
indexu 0, délka 9
Console.WriteLine(email.Substring(10));  // "example.com" – od
indexu 10 do konce

```

Přístup k jednotlivým znakům

`string` si můžete představit jako pole znaků typu `char` — každý znak má svůj index začínající od 0. Na rozdíl od skutečného pole ale znaky přes index pouze čtete, měnit je nelze (`string` je `immutable`). Typ `char` zapisujeme s jednoduchými uvozovkami: `'A'`, `'3'`, `' '`.

```

string slovo = "Praha";
Console.WriteLine(slovo[0]); // 'P'
Console.WriteLine(slovo[4]); // 'a'

// Procházení znaků
foreach (char c in slovo)
{

```

```

        Console.Write($"{c} ");
    }
    // P r a h a

```

Porovnávání řetězců

```

string a = "ahoj";
string b = "AHoj";

// ŠPATNĚ: == porovnává case-sensitive
Console.WriteLine(a == b); //
False

// SPRÁVNĚ: Porovnání bez ohledu na velikost písmen
Console.WriteLine(string.Equals(a, b,
StringComparison.OrdinalIgnoreCase)); // True

// Porovnání pro řazení
Console.WriteLine(string.Compare("alfa", "beta")); // záporné číslo
(alfa < beta)

```

Pro zvědavé — `StringBuilder`: Operátor `+` při každém spojení vytvoří nový řetězec v paměti. Pokud sestavujete řetězec z mnoha částí v cyklu, je to neefektivní. V takovém případě použijte `StringBuilder` ze jmenného prostoru `System.Text` — ten řetězec sestavuje postupně bez zbytečného vytváření mezivýsledků a je výrazně rychlejší.

Praktický příklad: Analýza textu

```

namespace AnalizaTextu;

internal class Program
{
    static void Main(string[] args)
    {
        Console.Write("Zadejte větu: ");
        string veta = Console.ReadLine();

        // Počet znaků včetně mezer
        Console.WriteLine($"Počet znaků: {veta.Length}");

        // Počet slov – rozdělíme podle mezer, prázdné položky
        // ignorujeme
        string[] slova = veta.Trim().Split(' ',
StringSplitOptions.RemoveEmptyEntries);
        Console.WriteLine($"Počet slov: {slova.Length}");
    }
}

```

```

// Počet samohlásek – procházíme znak po znaku
int pocetSamohlasek = 0;
foreach (char c in veta.ToLower())
{
    if ("aeiouáéíóúůý".Contains(c))
    {
        pocetSamohlasek++;
    }
}
Console.WriteLine($"Počet samohlásek: {pocetSamohlasek}");

// Věta velkými písmeny
Console.WriteLine($"Velkými písmeny: {veta.ToUpper()}");
}
}

```

Typ char — práce se znaky

```

char c = 'A';

// Statické metody třídy Char
Console.WriteLine(Char.IsLetter(c)); // True – je písmeno
Console.WriteLine(Char.IsDigit('5')); // True – je číslice
Console.WriteLine(Char.IsUpper('A')); // True – je velké
písmeno
Console.WriteLine(Char.IsLower('a')); // True – je malé písmeno
Console.WriteLine(Char.IsWhiteSpace(' ')); // True – je bílý znak

// Konverze
Console.WriteLine(Char.ToLower('A')); // 'a'
Console.WriteLine(Char.ToUpper('a')); // 'A'

// Numerická hodnota znaku (Unicode kód)
Console.WriteLine((int)'A'); // 65
Console.WriteLine((char)65); // 'A'

```

Shrnutí

- `string` je imutabilní — metody vracejí nový řetězec, původní nemění.
- Nejčastěji používané metody: `Length`, `ToUpper/ToLower`, `Trim`, `Contains`, `Replace`, `Split`, `Substring`, `IndexOf`.
- Pro porovnávání bez ohledu na velikost písmen použijte `StringComparison.OrdinalIgnoreCase`.
- Pro sestavování dlouhých řetězců v cyklu používejte `StringBuilder`.

Cvičení

1. Napište program, který zkontroluje, zda je zadané slovo palindrom (čte se stejně zleva i zprava), například „radar“.
2. Napište program, který načte větu a vypíše každé slovo na samostatném řádku s jeho délkou.
3. Napište program, který v zadaném textu spočítá výskyty každého písmene abecedy.

Kapitola 13: Doporučené zásady psaní kódu

Správné návyky v psaní kódu se formují od začátku — je mnohem snazší je získat hned, než je později přeučovat. Tato kapitola shrnuje základní zásady, které vám pomohou psát kód čitelný, přehledný a snadno opravitelný.

Platí jedno důležité pravidlo: kód čteme mnohem častěji, než ho píšeme. Váš kód bude číst kolega, vedoucí, nebo vy sami za půl roku, kdy budete mít detaily dávno zapomenuté.

Pojmenování

- **Proměnné a parametry:** `camelCase` — `platZamestnanec`, `pocetHracu`
- **Metody a třídy:** `PascalCase` — `VypoctiPrumer`, `class SpravceUzivatele`
- **Konstanty:** `PascalCase` — `MaxPocet`, `Pi` (méně časté, převzaté z C/Javy: `UPPER_CASE` — `MAX_POCKET`)
- Pojmenovávejte srozumitelně: `soucet`, ne `s`; `pocetZaku`, ne `pz`.
- Nepoužívejte zkratky, které nejsou obecně srozumitelné.

Odsazování a formátování

- Používejte odsazení **4 mezery** (nebo Tab = 4 mezery).
- Otevírací závorka `{` může být na konci předchozího řádku (K&R styl) — Visual Studio to tak formátuje standardně.
- Uzavírací závorka `}` vždy na vlastním řádku.
- Klávesová zkratka pro automatické přeformátování: **Ctrl+K, Ctrl+D**.

```
// Doporučený styl – odsazení jasně ukazuje strukturu
for (int i = 0; i < 10; i++)
{
    if (i % 2 == 0)
    {
        Console.WriteLine(i);
    }
}
```

Komentáře

- Pište komentáře k netriviálním částem kódu — proč, ne co.
- Vyhněte se zbytečným komentářům: `i++; // Zvýšení i o 1` — to je patrné z kódu.

- Užitečný komentář: `// Inicializace maxima prvním prvkem, aby fungovalo i pro záporná čísla`

Testování

- Testujte program s různými vstupy, nejen s těmi, pro které „určitě chodí“.
- Vždy testujte **hraniční hodnoty**: 0, záporná čísla, prázdný řetězec, první a poslední prvek pole.
- Před spuštěním si spočítejte očekávaný výsledek kalkulačkou a porovnejte s výstupem programu.
- Testujte i **neočekávané vstupy** — co se stane, když uživatel zadá text místo čísla, nebo stiskne Enter bez zadání hodnoty. Správné ošetření takových situací se naučíte postupně v dalších kapitolách, ale už teď je dobré si uvědomit, kde by váš program mohl selhat.

Postupný vývoj

Nepište celý program a pak ho spouštějte. Postup:

1. Napište malou část (například načtení vstupu).
2. Zkompilujte a spusťte — ověřte, že tato část funguje.
3. Přidejte další část.
4. Opakujte.

Tím předejdete situaci, kdy se v programu nahromadí desítky chyb najednou.

Časté zásady

- **Jeden příkaz na řádek** — usnadňuje krokování debuggerem.
- **Vždy {}** i pro jednořádkové bloky — zabrání chybám při přidávání řádků.
- **Neopakujte výpočty**: Pokud se stejný výpočet provádí vícekrát, uložte výsledek do proměnné.
- **Správný datový typ**: Pro celá čísla `int`, pro desetinná `double`, pro peníze `decimal`. Nedeklarujte vše jako `double`.
- **Inicializace**: Maximum a minimum inicializujte na první prvek pole, ne na 0.
- **Nerecyklujte proměnné** pro různé účely — matou čtenáře.

```
// ŠPATNĚ – stejný výpočet se opakuje třikrát, špatně se opravuje
Console.WriteLine($"Cena bez DPH: {cena * 100 / 121}");
Console.WriteLine($"DPH: {cena - cena * 100 / 121}");
if (cena * 100 / 121 > 1000)
    Console.WriteLine("Drahé zboží.");

// SPRÁVNĚ – výpočet na jednom místě, snadno se mění
double cenaBezDph = cena * 100 / 121;
Console.WriteLine($"Cena bez DPH: {cenaBezDph}");
Console.WriteLine($"DPH: {cena - cenaBezDph}");
```

```
if (cenaBezDph > 1000)
    Console.WriteLine("Drahé zboží.");
```

Shrnutí

Správné návyky v psaní kódu se formují od začátku. Čitelný a dobře strukturovaný kód je v profesionálním vývoji nutností — váš kód bude čist a upravovat i někdo jiný. Nebo vy sami za půl roku, kdy budete mít kód dávno zapomenutý.

Hlavní pravidla:

- Smysluplná jména, konzistentní odsazení, komentáře kde je to potřeba.
- Správné datové typy, žádné „recyklování“ proměnných.
- Testovat průběžně a s hraničními hodnotami.
- Program psát po částech, ne celý najednou.